
Dr. Pooja's Physio - E2E Test Run, In Execution Order

One pass, top to bottom, building its own data as it goes

Application

Dr. Pooja's Physio - Next.js 16 · React 19 · Supabase · Razorpay

Document version

1.0

Audience

A tester who has never used this application before

Environment

Throwaway Supabase project · Razorpay test mode · npm run dev

How to use it

Start at Step 1.1 and do not skip. Every value you need is on the step that needs it.

Contents

1. How to use this document
2. Part 1 - Start from nothing
3. Part 2 - Build the catalogue
4. Part 3 - Create the people
5. Part 4 - Onboard the first patient, end to end
6. Part 5 - Assign, join and deliver the session
7. Part 6 - The health record
8. Part 7 - The recommendation, and buying a programme
9. Part 8 - The home visit
10. Part 9 - The partner referral, end to end
11. Part 10 - The books
12. Part 11 - The four ways money comes off
13. Part 12 - Who may do what
14. Part 13 - The site itself, and the small things
15. Sign-off

1. How to use this document

1.1 What this is

This is **one test run, written in the order you perform it**. Start at Step 1.1, work down, and stop at the sign-off sheet at the end. Every screen, every value you type and every result you check is on the step that needs it.

It is deliberately **not** the reference plan. The companion document, *Complete Manual E2E Test Plan & Feature Guide*, is organised by area - all the patient cases together, all the admin cases together, with a test-data library at the front - which is the right shape for looking something up and the wrong shape for executing. Following it meant scrolling back to a table at the front to find out what to type, then scrolling forward again, over and over. Everything here is where you are.

The run builds its own data. You will reset the database at Step 1.2 and then create everything the run needs, in the order the application itself would have it created: the catalogue before anyone can book against it, the therapists before anyone can be assigned to one, the patient before there is a session to treat. Nothing is seeded for you, and nothing assumes a fixture that arrived from somewhere else. If a step needs a therapist with a roster, an earlier step made one.

That is also why the order matters more than usual. **Do not skip a step and do not reorder the parts.** A skipped step is usually a missing row three parts later, and it surfaces as a screen that looks broken.

1.2 What you need before Step 1.1

You need	Why	How to check you have it
The application running at <code>http://localhost:3000</code>	Everything	Open it. The home page renders with a teal splash.
A throwaway Supabase project	Step 1.2 empties it	Open the Supabase dashboard for the project and confirm nothing in it matters.
<code>ALLOW_DEBUG_DATA_RESET=true</code> in the server environment	Step 1.2	Without it, the reset route answers 404 , not 403.
One admin account that already exists	Step 1.2 keeps admins and deletes everyone else	Sign in at <code>/admin/login</code> . If you cannot, see Step 1.0.
Razorpay test keys	Every payment step	The Razorpay sheet says <i>Test Mode</i> across the top.
Google Chrome or Edge	A handful of steps read the DevTools console	F12 opens it.

If money can move for real, stop. Every payment step here uses Razorpay test mode. Check the key in the server environment starts `rzp_test_` before Step 4.6, not after.

1.3 The four conventions this run uses

Only four things are defined once rather than at the point of use, because they are true of every step rather than of one:

1. One password, everywhere you create an account. Every account you make in this run uses:

Where a step needs a *second, different* password - there is one, at Step 11.4 - it says so and gives you the value there.

2. Emails all end `@example.test`. `.test` is reserved and cannot be delivered to, so nothing you type here can reach a real inbox by accident.

3. Times are the clinic's. Every date and time the application prints is in **India Standard Time**, whatever your laptop is set to. When a step says "tomorrow at 4 PM" it means 4 PM as the screen shows it.

4. Calling a route without a terminal. Six steps ask you to call an API route directly, because this application enforces its rules twice - once in the screen and once in the route - and a screen that hides a button proves only the first. You do not need a terminal. Sign in as the person the step names, open DevTools (**F12**) → **Console** on any page of the app, paste, and press Enter:

```
const r = await fetch("/api/some/route", {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify({ some: "value" }),
});
({ status: r.status, body: await r.text() });
```

The request is same-origin, so the browser attaches that user's session cookie by itself - there is nothing to copy. Four things catch people out, and each step that matters repeats the relevant one:

- **For an anonymous call, use a private window.** Running it in a window where you are signed in sends your cookie and proves nothing.
- **For a different user, use a second browser or a second profile**, not a second tab - tabs share cookies.
- **`r.text()` rather than `r.json()`** when a route answers with no body, or `await r.json()` throws and hides the status you wanted.
- **Never paste the URL into the address bar.** That sends a GET, and these routes are POSTs; you will get **405** and think you found something.

1.4 How to record a result

Put a mark against every step as you go. The sign-off sheet at Step 13.1 asks for the totals.

Mark	Means
Pass	The expected result happened, all of it.
Fail	Any part of the expected result did not happen. Raise it with the template below.
Blocked	You could not run the step because something earlier failed. Name the step that blocked it.
N/A	The step's feature is switched off in this environment on purpose. Say which setting.

Raise a failure like this. The two lines people leave out are the last two, and they are the two that decide whether anyone can act on it.

Step: 4.6
What I did: Paid for the Back & Spine consultation with card 4111 1111 1111 1111
What I expected: Confirmation screen, session listed as Pending under Your Sessions
What happened: Razorpay sheet closed, spinner ran for ~30s, screen stayed on step 3
Screenshot: yes - attached
Console errors: "POST /api/razorpay/verify 500" - copied in full below
Reproducible: 3 times out of 3
Who was signed in: QA Patient A (qa.patient.a@example.test)

1.5 Severity, so a failure lands in the right queue

Severity	Means	Examples from this run
P0	Money, clinical data, or access control is wrong. Stop and report immediately.	A patient charged a figure they were not quoted; one patient seeing another's record; a non-admin reaching an admin route.
P1	A main journey is broken or badly misleading, with no workaround.	Booking cannot complete; a recommendation never reaches the patient.
P2	Something is wrong but has a workaround.	A count disagrees with the list under it; an empty state reads badly.
P3	Cosmetic.	Spacing, a wrapped label, a stale tooltip.

2. Part 1 - Start from nothing

What this part does. Empties the database, proves the wipe's own guards, and confirms the one account that has to survive it did. Everything after this part builds on an empty database, so nothing here is optional.

Time. About 15 minutes.

Step 1.0 - Make sure you can sign in as an admin

The reset keeps admin logins and deletes everyone else, so you need one admin **before** you press it. On a database nobody has used yet there is exactly one, created by hand.

Do this

1. Open `http://localhost:3000/admin/login`.
2. Tap **Email Address** and enter your admin's email. If you are creating one now, use:

Field	Value
Email	<code>qa.admin@example.test</code>
Password	<code>QaTest!2024pass</code>
Full name	<code>QA Master Admin</code>

1. Tap **Password**, enter the password, and tap **Sign In**.

Expect. You land on `/admin/dashboard`. The sidebar brand reads **Master Admin** above *Admin Panel*, and the eyebrow above the page heading reads **Master Admin** as well - twice, on purpose, because the sidebar collapses on a phone. You can see all seven sections: **Today, Sessions, People, Money, Catalog, Logs, Settings**.

If you cannot sign in. The account has to be made in the Supabase dashboard: **Authentication → Users → Add user** with that email and password and *Auto Confirm User* ticked, then **Table Editor → profiles**, find the row, and set `role` to `admin`, `active` to `true`, `admin_scope` to `full`. There is no screen in the application that creates the first admin, deliberately.

A Master Admin is what this run calls a `full` scope admin. Three narrower desks - Operations, Finance, Clinical - are created at Step 3.7. Until then, every admin instruction means this account.

Step 1.1 - Confirm the debug bar is there

Do this. Look at the top of the page.

Expect. A black **Debug** bar pinned across the top, listing routes and carrying a **Reset data** button. It is on in every environment on purpose until launch, so seeing it here is correct, not a defect.

If it is missing, `NEXT_PUBLIC_SHOW_DEBUG_NAV` is set to exactly `false` in the environment. Nothing else in this run works without it - unset that variable and restart the server.

Step 1.2 - Empty the database

Do this

1. In the Debug bar, tap **Reset data**.
2. Read the red warning that appears.
3. Tap the confirmation field - its placeholder reads `RESET ALL DATA` - and type, deliberately wrongly:

```
reset all data
```

1. Look at the **Reset** button without tapping it.
2. Clear the field and type it exactly:

```
RESET ALL DATA
```

1. Tap **Reset** and wait for the button to stop reading **Resetting...**

Expect

- At step 2, the warning names both survivals: *"Deletes people, sessions, purchases, money and settings. Admin logins and your conditions (with their programmes) survive - the rest of the catalog does not. No undo."*
- At step 4, the **Reset** button is **disabled and visibly faded**. A lower-case phrase never arms it.
- At step 6, a teal confirmation appears **with real figures** - *"N accounts deleted, M admins kept"*. **Two zeroes is a P0**: it means the route read a field name the database function does not return, and a wipe that emptied everything is indistinguishable from one that did nothing.
- You are **still signed in**. The reset does not destroy your session.

Two specific failures worth knowing by name

What you see	What it is
UPDATE requires a WHERE clause	A statement inside the reset function is missing its WHERE . This can only ever be caught by pressing this button - applying the schema or running the statement in the SQL editor connects as a different database role that does not enforce it. P0.
404 in the console, or nothing happens at all	ALLOW_DEBUG_DATA_RESET is not set in the server environment. The route answers 404 rather than 403 deliberately, so its existence is not confirmed to a stranger. Not a defect.

Step 1.3 - Check what the reset kept and what it removed

Do this. Visit each of these and look.

Go to	Expect
People → Patients	An empty-state message, not a table.
Sessions → All Sessions	No sessions.
Catalog → Conditions	Whatever conditions existed before the reset are still here, at their prices and in their order. This is the one part of the catalogue the wipe keeps, because it is the part an admin builds by hand.
Catalog → Service Areas	Empty. Service areas are cleared.
Catalog → Packages, home-visit section	Empty. Home-visit packages are cleared.
Logs → All Activity	Exactly one row - the reset recording itself. The wipe truncates the log and then writes that row, so a completely empty log here means the most destructive action in the product went unattributed. P1.
Settings → User Access	At least one admin, including your own row.
Today → Risk	An empty queue.
/conditions (public, signed out)	The conditions still render. The public site must not come back empty - that reads as the clinic having shut rather than as test data being cleared.

If Settings → User Access is empty, stop the run and restore from a backup. The reset must never leave the clinic with nobody who can open the back office.

Optional, if you have the Supabase SQL editor open. Run each and expect 0:

```
select count(*) from communication_flags;
select count(*) from risk_signals;
select count(*) from appointments;
```

And confirm the risk thresholds came back rather than being wiped:

```
select rule_key, enabled from risk_rules order by rule_key;
```

Eight rows, with `plan_conversion_low` and `post_consultation_dropout` **disabled** - those two need a clinic baseline nobody has yet, so they ship off.

Step 1.4 - Note the two conditions the reset left you

Because conditions survive, you may already have some. Look at **Catalog → Conditions** and write down what is there.

- **If the list is empty**, you will create all three at Step 2.1 and the run proceeds exactly as written.
- **If rows already exist**, you will still create the three this run needs at Step 2.1. Leave the others alone; they cost nothing, and deleting them is its own test at Step 13.4.

3. Part 2 - Build the catalogue

What this part does. Creates everything a patient can be sold: three conditions, three programmes, two home-visit packages and two service areas. Nothing before this point can be booked, because there is nothing to book against.

Who you are. Signed in as the Master Admin from Step 1.0, at `/admin/dashboard`.

Time. About 45 minutes.

Step 2.1 - Create the three conditions

You need three, and the run refers to them by name from here on. Create them one at a time.

Do this for each row below

1. Open **Catalog → Conditions**.
2. Tap the create control.
3. Fill the five fields exactly as the table gives them.
4. Leave the cover image alone for now - Step 2.2 does that.
5. Save.

Condition 1

Field	Value
Category Name	QA Back & Spine Care
Price (₹)	1999
Session Length (min)	60
Order	1
Button Text	Book Assessment

Condition 2

Field	Value
Category Name	QA Knee & Joint Care
Price (₹)	1799
Session Length (min)	45
Order	2
Button Text	Book Assessment

Condition 3

Field	Value
Category Name	QA Neuro Rehabilitation
Price (₹)	2499
Session Length (min)	60
Order	3
Button Text	Book Assessment

Expect, after all three are saved

- All three are listed in **Catalog → Conditions** in order 1, 2, 3.
- Open `/` and `/conditions` in another tab **now**, not in five minutes. All three render immediately. These pages are cached for five minutes, and the save is supposed to clear that cache - **a condition that only appears after a wait is a defect**, because it reads as a save that silently failed and gets made twice.
- Open `/book`. The concern dropdown lists `QA Back & Spine Care - ₹1,999 / 60 min` and the other two, with the price and length formatted exactly like that.
- A condition with no cover shows a **tinted placeholder panel at the same height** as a photograph would be - never a broken-image icon.

Now prove the form refuses bad input. Open the create form again and try each of these, one at a time. None of them should save.

What you type	Expected message
Category Name blank, everything else filled	Missing title, priceInr, or durationMinutes
Price 0	Price must be a positive number
Price -100	Price must be a positive number
Price abc	Price must be a positive number
Session Length 0	Session length must be a positive number of minutes
Order xyz	Order must be a number

Close the form without saving when you are done.

Step 2.2 - Put a cover on a condition, and position it

You need a picture. Any landscape JPG or PNG **whose subject is well off to one side** - that is the whole point of the step. A centred photograph proves nothing here. Save it somewhere you can find it.

Do this

1. Open **Catalog → Conditions** and edit `QA Back & Spine Care`.
2. Tap **Upload an image** and choose your file.
3. Read the row that replaces the drop zone.
4. Tap **Preview & position**.
5. Drag the picture until the subject sits where you want it, then tap **Done**.
6. Save the form.

Expect

- At step 3, the drop zone becomes a row naming the file, with **Preview & position**, **Replace** and **Remove**, reading `Centred - not positioned yet` until you move it.
- At step 5, **the frame stays still and the picture moves inside it** - dragging right moves the picture right. A rule-of-thirds guide sits over it and the focal point shows as two percentages.
- The three small frames below the picture - **Card**, **Square**, **Dialog** - all move **together** as you drag. This is the entire feature: one position that is correct in every shape. A position right in one frame and wrong in another is a **P0**.
- **Done** only stages it. Nothing on the public site changes until you save the form at step 6, and then it changes immediately.

Then check the same position holds in all three real frames. Open, in turn:

Where	Expect
<code>/conditions</code>	The card crops it 4:3 with your subject where you put it.
That condition's View full details dialog	Cropped 16:9, subject still where you put it, and the heading sits on its own band below the picture - nothing is written over the photograph.
<code>/patient/dashboard/book</code> , signed in as a patient (you will have one at Step 4.2 - come back then, or take it on trust now)	The same card as the public pages: cover, chips, ticks, price and a Book button. It must not be a text-only list.

Now the two refusals. Back on the form, try to upload:

File	Expected message
A PDF, or an SVG	<code>Please upload a JPG, PNG or WebP image.</code>
Any image larger than 5 MB	<code>That image is larger than 5 MB. Please use a smaller file.</code>

Step 2.3 - Create the three programmes

A programme is two or more sessions. **None of these can be bought from a price list** - that is the rule, not a limitation of the data. They reach a patient only through a therapist's recommendation, which you will see happen at Part 7.

Do this for each. Open **Catalog → Packages**, tap create, fill in the column, save.

Field	Programme P1	Programme P2	Programme P3
Category	QA Back & Spine Care	QA Neuro Rehabilitation	QA Back & Spine Care
Package Name	QA Spine Recovery 6 Sessions	QA Neuro Rehab 8 Sessions	QA Single Session
Subtitle	Six weeks, one therapist, measured progress	Eight sessions of gait and balance work	One assessment
Description	A structured six-session block for persistent lower-back pain.	An eight-session neurological rehabilitation block.	A single 60-minute session.
What We Promise (one per line)	The same therapist every session / A written home programme / Progress measured, not guessed	The same therapist every session / Gait retraining / Family guidance	A full assessment
Sessions Included	6	8	1
Bundle Price (₹)	9999	17999	1999
Compare-at Price (₹)	leave blank	leave blank	leave blank
Therapist Pay Basis	Discounted package price	Category list price	Discounted package price
Validity (days)	90	120	30
Session Duration (min)	leave blank	leave blank	leave blank
Minimum gap between sessions (hours)	24	48	leave blank
Maximum sessions per week	3	2	leave blank
Maximum purchases per patient	2	1	leave blank
Display Order	1	2	3
Active	ticked	ticked	ticked

Expect

- All three save.
- **The Category is locked after creation.** Re-open P1 and confirm you cannot change it - live purchases point at it.
- On the public site, a programme appears as a card **with no Buy button**, reading instead "Arranged by your therapist after your first session." If any programme here can be bought directly from a public page, that is a **P0** - treatment volume is never sold before an assessment.

- P1 and P3 are offered to a therapist recommending for **Back & Spine** only; P2 for **Neuro Rehabilitation** only.

Now the five refusals. On the create form, try each:

What you type	Expected message
Package Name blank	Package Name is required.
Sessions Included 1	Sessions Included must be a whole number of 2 or more.
Bundle Price 0	Bundle Price must be a positive number.
Compare-at 5000 with Bundle 9999	Compare-at Price can't be lower than the Bundle Price.
Display Order abc	Order must be a number.

P3 has one session and saved anyway. That is correct and is not a contradiction of the rule above: one session is a consultation, and there is nothing to assess before selling somebody a single appointment. The "2 or more" refusal is on the *Sessions Included* field of a **programme** form; P3 is created as the single-session row the same screen allows.

Step 2.4 - Switch home visits on

Home visits are off by default, and six surfaces change when you switch them on. Check them **before** as well as after, or you cannot tell the switch did anything.

First, with it still off, open each of these and note what you see:

Where	Expect while off
/home-visit	404. Not an empty page - a 404.
The public header nav	No Home Visit entry.
The footer's Explore column	No Home Visit entry.
The home page's connector grid at the bottom	No Home Visit tile.
Any inner page's "Where to go next" strip	No Home Visit tile.
/book-home-visit	Refused.

Now switch it on

1. Open **Settings → Programmes & Home Visits**.
2. Find **Home Visit** and switch it **on**. Save.

Expect. A confirmation toast naming the thing and its new state - *"Home visits are on"*, not a bare "Saved". Then re-check all six surfaces above: every one of them now has the Home Visit entry, and /home-visit renders.

Why the off state matters. While the switch is off the page 404s, so every list that mentions it has to drop the entry rather than link into a dead end. A Home Visit link that is visible while the page 404s is a **P1**.

Step 2.5 - Create the two home-visit packages

Open **Catalog** → **Packages** and use the home-visit section.

Field	HV1	HV2
Package Name	QA Home Visit - Single	QA Home Visit Recovery - 4 Visits
Subtitle	One visit at your door	Four visits over a month
Description	A single home assessment.	A four-visit home rehabilitation block.
Benefits (one per line)	A physiotherapist at your door / Full assessment	The same therapist each visit / Family training
Visits Included	1	4
Package Price (₹)	2499	8999
Visit Duration (minutes)	60	60
Validity (days)	30	90
Minimum gap between visits (hours)	leave blank	48
Maximum visits per week	leave blank	2
Travel fee included in price	unticked	unticked
Lock to one therapist	ticked	ticked
Active	ticked	ticked

Expect

- Both save and appear on `/home-visit`.
- Only HV1 is offered with a Book button.** HV2 has four visits, so it is a programme: it can be recommended but not bought. You will prove the route refuses it directly at Step 11.6.

Step 2.6 - Create the two service areas

A home visit cannot be sold anywhere until a pincode is covered.

Do this. Open **Catalog** → **Service Areas** and create both.

Field	Area 1	Area 2
City	Bengaluru	Bengaluru
Area name	Indiranagar	Koramangala
Travel fee (₹ per visit)	150	200
Pincodes	560038	560095
Notes	Core service area	Second phase

Expect. Both save.

Now the refusals. Try to create a third area and check each message:

What you type	Expected message
A third area containing 560038	Another service area already covers that pincode. (or Every pincode in that list is already a service area.)
Pincodes left blank	Enter at least one pincode.
Pincode 0560038 (leading zero)	Enter a valid 6-digit pincode.
Pincode 56003 (five digits)	Enter a valid 6-digit pincode.
City blank	City is required.
Travel Fee -50	Travel Fee must be zero or a positive number.

Write these two down - later parts use them:

- `560038` is **serviceable**, travel fee **₹150 per visit**.
- `560025` is **not serviceable** and is the address you will use to test the waitlist at Step 8.7.

Step 2.7 - Confirm the settings this run assumes

You are not changing these yet - you are checking where they stand, so that a later result is not a mystery.

Setting	Where	Should read	Why it matters here
Therapist-Suggested Sessions	Settings → Programmes & Home Visits	on	Part 7 needs it. On a database that predates the change it may still be off - switch it on now if it is.
Assign a Therapist Automatically	Settings → Programmes & Home Visits	off	Parts 4 and 5 expect a paid session to wait in the admin queue. Step 12.4 switches it on deliberately.
Session Balances From The Ledger	Settings → Programmes & Home Visits	off	Step 10.8 flips it and checks the balances still agree.
The clinic approves a recommendation	Settings → Programmes & Home Visits	on	Part 7 is written around the review queue.
First session offer	Settings → Offers & Discounts	off	Part 4 expects the patient to be charged list price. Step 9.1 switches it on.
Promo codes	Money → Costs	off	Step 9.3 switches it on.
Patient invites	Settings → Offers & Discounts	off	Step 9.5 switches it on.
Online Booking Lead Time	Settings → Booking Rules	12 hours	Every booking step assumes 12.
Online Cancellation Refund Window	Settings → Booking Rules	24 hours	Step 10.2 depends on it.

Also check, on any Settings screen you opened: under the page heading there are **two lines** - one saying what the screen is in plain words, and a second starting "**For example:**". A Settings screen whose heading is followed by "How the product behaves" and nothing else has lost its own description, which is a **P2**.

Step 2.8 - Choose which four lead the home page

The home page does not list every condition - it leads with **four** and sends the reader on for the rest. Which four is an admin's choice, not a count of what sells: a home page that rearranges itself when a booking lands has changed without anybody deciding, and a condition added today could never reach it until it had already sold.

First, look at it with nothing ticked. Open [/](#) and read the "what we treat" band.

Expect. Your first four conditions, in order, with a link on to </conditions> for the rest. **An empty band is a P1** - nothing is ticked yet, and the fallback exists precisely so the band is never empty while somebody has not opened the screen.

Now choose. In **Catalog → Conditions**, tick the feature control on [QA Neuro Rehabilitation](#) and untick the others. Save, then reload [/](#).

Expect

- Only the ticked one leads, and the band still links on to </conditions>, which shows **everything**.
- The cap is **stated on the screen that sets it**, so you know how many will show before you reload the site.
- Ticking more than the cap is **not an error** - it simply shows the first of them.

Then do the same on Home Visit. Tick one home-visit package and reload </home-visit>.

Expect. The page shows the ticked one and **reveals the rest in place** rather than linking somewhere - it *is* its own full list and has nowhere else to send anybody. A control that opens a list identical to the one above it is a dead end with a label on it.

And check the two controls are not the same control. On the home-visit form there are two: one picks whether it leads the page, the other is **Highlight with a ring**, which is styling. They must read as different things. Two controls both called "Feature", meaning different things, in the one place an admin meets both, is a **P2**.

One more thing to check. The patient dashboard's **Book a Session** screen still shows **everything**, ticked or not. That is the screen somebody opens *to* book, so trimming it hides what they came for.

4. Part 3 - Create the people

What this part does. Puts three therapists, one partner hospital and three scoped admins into an empty database, each through the door the application actually uses. Two of them self-register and wait for you to approve them; the hospital and the admins are minted from the back office.

Time. About 50 minutes.

Step 3.1 - Therapist A applies

Who you are. Signed out. Use a private window if your admin session is open in this browser.

Do this

1. Open `/therapist/login`. Confirm there are two tabs: **Sign In** and **Apply to Join**.
2. Tap **Apply to Join**.
3. Fill it in:

Field	Value
Full Name	QA Therapist A
Email Address	qa.therapist.a@example.test
Phone	+91 90000 10001
Qualifications & License / Council Reg No.	MPT (Ortho), KSCP Reg 44821
Password	QaTest!2024pass
Confirm Password	QaTest!2024pass

1. Tap **Submit Application**.

Expect

- The button reads `Submitting...`, then the form returns to the **Sign In** tab with a confirmation that the application is with the clinic.
- **No "check your email" step appears anywhere.** Email confirmation is off in this product by design, and the admin's approval is the only gate. A screen telling this therapist to check their inbox is a **P1** - it sends them to wait for something that will never arrive.

Step 3.2 - Prove an unapproved therapist is held at the door

This is the first of the six console steps. Read §1.3 if you have not.

Do this

1. Still on `/therapist/login`, **Sign In** tab, sign in as `qa.therapist.a@example.test` / `QaTest!2024pass`.
2. Note where you land.
3. With that same session, open DevTools (**F12**) → **Console** and run:

```
const r = await fetch("/api/therapist/save-availability", {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify({ slots: [{ dayOfWeek: 1, hour: 10 }] }),
});
({ status: r.status, body: await r.text() });
```

Expect

- Step 2 lands on `/pending-approval`, not the dashboard.
- Step 3 returns **403** with `Your account is not active - it is either awaiting admin approval or has been suspended.`

- **The second one is the one that matters.** The screen refusing is good; the route refusing is the point. A valid session cookie must not be able to call around the interface. A **200** here is a **P0**.

Sign out.

Step 3.3 - Therapists B and C apply

Repeat Step 3.1 twice more. You need all three: B proves one therapist cannot see another's patients, C is the one you put on leave.

Field	Therapist B	Therapist C
Full Name	QA Therapist B	QA Therapist C
Email Address	qa.therapist.b@example.test	qa.therapist.c@example.test
Phone	+91 90000 10002	+91 90000 10003
Qualifications & License	MPT (Neuro), KSCP Reg 44822	BPT, KSCP Reg 44823
Password	QaTest!2024pass	QaTest!2024pass

Expect. Both applications submit the same way. Do not approve anyone yet.

Step 3.4 - Approve all three, as the admin

Who you are. Signed in as the Master Admin.

Do this

1. Open **Today** → **Approvals**.
2. Read the badge on that tab and the queue itself.
3. Approve QA Therapist A.
4. Approve QA Therapist B and QA Therapist C.

Expect

- All three applications are listed **oldest first**, each aged in words (12 minutes ago), not stamped with a date.
- The queue's own age is the age of the **oldest** waiting row, and it does **not** reset when the screen refreshes. Watch it across one refresh: an item that jumps back to *just now* is a **P1** - a signup that has waited three days would read as having just arrived, which is backwards for the one kind of item that gets more urgent the longer it waits.
- After approving, each therapist disappears from the queue and the badge falls by one.
- Open /team in another tab: all three now appear there, immediately. A therapist approved but missing from /team for five minutes means the approval did not clear that page's cache - **P2**.

Then sign in as Therapist A (qa.therapist.a@example.test / QaTest!2024pass) and confirm the sign-in now lands on /therapist/dashboard , with the sidebar showing **Overview, Availability, Sessions, Earnings, My Patients, Edit Profile**, and **Back to Home** at the foot of the nav.

Step 3.5 - Set what each therapist is paid

A therapist's revenue share decides every payout figure later in this run, so it has to exist before anybody delivers anything.

Do this. As the admin, open **People → Therapists**, open each therapist, and set:

Therapist	Revenue share %	Home-visit revenue share %
QA Therapist A	60	65
QA Therapist B	55	leave unset
QA Therapist C	50	leave unset

Expect

- Each saves and the therapist's own Overview header then reads `Your Revenue Share: 60%` (and so on) when they next open it.
- Leaving B and C's home-visit share unset is deliberate.** Part 8 checks that a home visit delivered by B falls back to their ordinary 55%, rather than to zero or to A's 65%.
- `-5` and `150` are both refused with `Revenue share must be a number between 0 and 100`.

Step 3.6 - Give Therapist A a roster

The roster is the clinic's planning record - who can be *offered* work. It does **not** filter what times a patient is offered at booking; you will prove that at Step 12.5.

Do this

- Open **Sessions → Roster**.
- Read the landing view before opening anybody.
- Open `QA Therapist A`.
- Set the weekly schedule to these working periods:

Day	Periods
Monday to Friday	<code>09:00-13:00</code> and <code>14:00-18:00</code>
Saturday, Sunday	none

- Save.
- Now add a date exception: pick **the next Tuesday at least a week away**, set it to `14:00-18:00` only, reason `Clinic audit in the morning`. Save.
- Look at the Tuesday **after** that one, and at the weekly template itself.

Expect

- Step 2: the Roster opens on a **list of therapists** with a summary of what each works and their leave state - not on a calendar date, and not on an eighteen-column hourly grid.
- Step 3: the **period editor** opens. You set working periods, never individual hours.
- Step 7: **only the one date changed**. Every other Tuesday still shows `09:00-13:00` and `14:00-18:00`, and the weekly template is untouched. An exception replaces that whole day and nothing else; a change that leaks into the template is a **P0**.

Now set Therapist C on leave. In the Roster, put `QA Therapist C` on leave for a five-day range starting next Monday, reason `Annual leave`.

Expect. The roster shows them off. **Their weekly schedule is untouched and is still there when the leave is removed** - there is nothing to restore on the way back because nothing was removed.

And give Therapist B a roster too, since Part 8 needs them bookable: Monday to Friday `10:00-16:00`.

Step 3.7 - Create the three scoped admins

Four desks exist. You have the first; these are the other three, and the run needs them from Part 11 onwards.

Do this. Open **Settings → User Access**. For each row below, use the **Account type** picker and create the account.

Full name	Email	Account type to pick
<code>QA Admin Operations</code>	<code>qa.admin.ops@example.test</code>	Operations
<code>QA Admin Finance</code>	<code>qa.admin.finance@example.test</code>	Finance
<code>QA Admin Clinical</code>	<code>qa.admin.clinical@example.test</code>	Clinical

Expect, on the picker itself. It lists **six** options in two groups - **Clinic** (Patient, Therapist) and **Back office** (Master Admin, Operations, Finance, Clinical). There is no "Admin" entry that then reveals a second *Access level* dropdown. Picking the desk by name is the whole control.

Write the password down the moment it appears. Each of these three gets a **generated one-time password, shown once on this screen** - not `QaTest!2024pass`. It is deliberately never emailed and never written to the activity log. If you lose one, you cannot recover it: set a new password in the Supabase dashboard under **Authentication → Users**, or delete the account there and create it again here.

Then check it survived the screen. Stay on User Access for thirty seconds without touching anything.

Expect. The password is **still on screen**. Creating the account writes a `profiles` row, which fires a realtime refresh, and the password used to be swept off the screen mid-sentence by that refresh. If it disappears on its own, that is a **PO** - the credential is then unrecoverable.

Step 3.8 - Read the access matrix

Do this. Still on **Settings → User Access**, scroll to the matrix.

Expect

- Rows are the jobs people describe; columns are the four desks. Every cell says `none`, `view` or `manage`.
- **The cells are not checkboxes.** They are a description of what the routes enforce, not switches - a tick you could toggle that did not change a route would be a lie.
- Exactly one grant reads **view: Finance reads Sessions**. Finance can read what a ₹1,200 session was for, and cannot cancel the sessions they are reconciling.
- **Logs** is `manage` for Master Admin and **`none` for all three other desks**.

Step 3.9 - Prove the reset refuses a narrower admin

You could not run this at Part 1 because the account did not exist yet. It exists now.

Do this

- 1. Sign out of the Master Admin.
- 2. Sign in at `/admin/login` as `qa.admin.ops@example.test` with the one-time password from Step 3.7.
- 3. In the Debug bar, tap **Reset data**.
- 4. Type `RESET ALL DATA` in the confirmation field.
- 5. Tap **Reset**.

Expect. A red error in the bar reading exactly:

```
Only a Master Admin can reset data.
```

Nothing is deleted. Go back to **People → Therapists** as the Master Admin and confirm all three therapists are still there. If this call emptied the database, stop the run - it is the most serious defect this document can find.

If you get `Invalid login credentials` here, that is the generated password, not the reset. Re-read the warning in Step 3.7.

Sign back in as the Master Admin.

Step 3.10 - Provision the partner hospital

A hospital never self-registers into a working account. The public page collects an enquiry; an admin converts it.

First, submit the enquiry. Sign out (or use a private window) and open `/hospitals`. Scroll to the enquiry form and submit:

Field	Value
Organisation name	QA Sunrise Hospital
Contact person	QA Hospital Admin A
Email	qa.hospital@example.test
Phone	+91 80400 10001
Address	18 Airport Road
City / State / PIN	Bengaluru / Karnataka / 560017

Expect. A confirmation appears. **No account is created and no login works yet** - an enquiry is not a partner. Try signing in at `/hospital/login` with that email and confirm it fails.

Then convert it. As the Master Admin, open **People → Partners**, find the lead `QA Sunrise Hospital`, and tap the onboard control:

Field	Value
Organisation Name	QA Sunrise Hospital
Contact Person	QA Hospital Admin A
Email	qa.hospital@example.test
Revenue Share %	10

Expect

- The account is created, and the screen shows **a generated password and a generated referral code, once. Write both down.** Part 9 needs the referral code.
- Open **Logs → All Activity**. The onboarding is recorded, naming who onboarded whom and when - **and the password is not in it**. Every admin can read that log, so a generated password there would be a credential leak. Finding one is a **P0**.
- A revenue share of `-5` or `150` is refused with `Revenue share must be a number between 0 and 100`. Re-submitting the same email is refused rather than creating a second account.

Then sign in as the hospital at `/hospital/login` and confirm you land on `/hospital/dashboard`, sidebar reading **Overview, Refer a Patient, Your Referrals, Earnings, Edit Profile**. The money word here is **Earnings** - the same word the therapist's sidebar uses. "Revenue & Payouts" or any third name for the same thing is a **P2**.

Step 3.11 - Checkpoint

Before going on, confirm all of this is true. Part 4 assumes every line.

	Should be
Conditions	Three, priced ₹1,999 / ₹1,799 / ₹2,499
Programmes	P1 (6), P2 (8), P3 (1)
Home-visit packages	HV1 (1 visit), HV2 (4 visits)
Service areas	<code>560038</code> at ₹150, <code>560095</code> at ₹200
Therapists	A, B, C - all approved, shares 60/55/50, A and B rostered, C on leave
Hospital	<code>QA Sunrise Hospital</code> provisioned, referral code written down
Admins	Master Admin plus Operations, Finance, Clinical - all three passwords written down
Patients	None yet. Part 4 makes the first one.

5. Part 4 - Onboard the first patient, end to end

What this part does. Takes one person from never having heard of the clinic to a paid, confirmed session in the diary - registering, being approved, booking, paying, and landing on their own dashboard. Everything later in the run hangs off this patient.

Who you are. Mostly **QA Patient A**. Two steps need the admin; each says so.

Time. About 60 minutes.

Use a second browser for the admin. You will switch between the patient and the admin repeatedly, and tabs in one browser share cookies - signing in as the admin in a second tab signs the patient out of the first. Use Chrome for the patient and a private window (or a second profile) for the admin.

Step 4.1 - Meet the clinic as a stranger would

Before registering anything, look at what a visitor sees. This is also the fastest check that Part 2's catalogue actually published.

Do this. Signed out, visit each page and check the one thing named.

Page	Check this
/	A teal splash appears once , then the home page. Four conditions lead the "what we treat" band, not all of them, with a link on to the rest.
/conditions	All three of your conditions, in order 1, 2, 3, at ₹1,999 / ₹1,799 / ₹2,499.
/how-it-works	Renders, ends with the same closing band as every other page.
/home-visit	Renders (you switched it on at Step 2.4). HV1 has a Book button; HV2 does not.
/team	All three therapists from Part 3.
/mission	The mission and vision sentences, the promises band, the limits band.
/faq	Renders.
/hospitals	The enquiry form you used at Step 3.10.

Also check the splash behaves. Reload / - the splash should **not** show again. Open a brand-new tab to / - it **should**. A splash on every navigation is a **P2**; a splash that covers a page you are already reading is worse, because the same behaviour over a checkout in progress is what it must never do.

And check no programme is on sale. Nowhere on / or /conditions is there a price list of programmes with a Buy button. Each programme card reads *"Arranged by your therapist after your first session."* A programme purchasable from a public page is a **P0**.

Step 4.2 - Register Patient A through the booking wizard

This is the commonest real path: a stranger books and registers in one screen. The account, the booking and the payment all happen on /book .

Do this

1. From /conditions , tap **Book Assessment** on QA Back & Spine Care . You land on /book .
2. **Step 1 of the wizard.** Read what is pre-selected before changing anything:
3. a date, a time and a language are already chosen for you.
4. the detected timezone is stated on screen.

5. Choose **tomorrow**, and an hour of **4:00 PM**.
6. Leave the language as the first chip.
7. Tap **Continue**.
8. **Step 2**. Fill the account fields:

Field	Value
Full Name	QA Patient A
Email Address	qa.patient.a@example.test
Password	QaTest!2024pass
Confirm Password	QaTest!2024pass
Phone	+91 98765 43210
Referral Code	leave blank

1. Under **What would you like help with?**, pick QA Back & Spine Care .
2. In **Anything else we should know?**, type exactly:

Desk job, pain worse after sitting all day. Goal: sit through a full workday.

1. Tick the telehealth consent box.
2. Tap **Continue**.
3. **Step 3**. Read the whole screen before tapping anything. Note the figure on the button.

Expect, at each step

- **Step 1's dates**. Today is usually **not** offerable - the booking lead time is 12 hours, so at 10 AM the first bookable slot is 10 PM tonight, and after about midday today drops off the calendar entirely. That is the rule working, not a bug. The boundary day is **partly** available: early hours greyed, later ones live.
- **Changing the date re-picks the hour**. Pick a different date and the hour resets to that day's earliest eligible one. Your old pick is not carried across, because it might not clear the lead time on the new date.
- **Step 3 quotes what it will charge**. The Session Fee reads **₹1,999** - the category price - and the button reads **Request Booking**. With no discount running, the figure on the button and the figure Razorpay opens with must be **the same number**. They were not, once: the wizard printed the category price while checkout quietly applied an offer behind it. **A payment screen showing one figure and charging another is a P0.**
- **The cancellation line** reads *"Free cancellation up to 24 hours before your slot"* - the admin setting from Step 2.7.
- **The exit link** at the foot, outside the wizard, reads **Back to Home** and goes to  while you are signed out.

Do not tap Request Booking yet. Step 4.3 checks two refusals first.

Step 4.3 - Check what the wizard refuses

Do this, one at a time, returning to Step 1 between attempts.

Try this	Expect
On Step 2, email <code>not-an-email</code>	An invalid-email message. No account created.
Password <code>abc12</code> (five characters)	A minimum-length message. No account created.
Confirm Password <code>QaTest!2024pas</code> (one character short)	A passwords-do-not-match message.
Phone <code>12345</code>	An invalid-phone message.
Referral Code <code>ZZZZZZ</code>	<code>Code not recognized - double-check it or leave blank</code> in red.
Leave the consent box unticked and tap Continue	Refused, with a readable message.
Leave What would you like help with? unset	Refused.

In every case: the form stays on screen with what you typed intact, the message is in plain English, and **no stack trace, database column name or row id appears anywhere**. A raw Postgres string such as *"new row violates row-level security policy"* on this screen is a **P0** - that exact failure is what the current booking route exists to prevent.

One more, in a second browser. Sign in as **QA Therapist A** and open `/book`.

Expect. Not the form - a **wrong account** panel, saying a clinician wanting therapy should sign out and use a separate patient account. Repeat signed in as the hospital (it points at referring) and as the admin (it points at the back office's own booking screen). **This is checked before every other branch**, so it shows even with a `?package=` on the URL.

Step 4.4 - Create the booking and pay

Back in the patient's browser, refill Step 1 and Step 2 exactly as Step 4.2 gave them, and reach Step 3.

Do this

1. Tap **Request Booking**.
2. The Razorpay sheet opens. Confirm it says **Test Mode**.
3. Pay with the test card:

Field	Value
Card number	<code>4111 1111 1111 1111</code>
Expiry	any future date, e.g. <code>12/30</code>
CVV	any three digits, e.g. <code>123</code>
Name	<code>QA Patient A</code>
OTP / 3DS page	choose Success

Expect

- The figure in the Razorpay sheet is **₹1,999** - the same figure the button quoted.
- After paying, Step 3 is replaced by a **Payment Confirmed** panel with a **Go to Dashboard** link.
- Tap it. You land on `/patient/dashboard` **without being bounced to `/pending-approval`** - genuinely attempting checkout approves the patient, deliberately on the attempt rather than on success, so

somebody whose card fails three times still reaches their dashboard rather than a waiting screen.

- The session is listed, **Pending** - paid, but nobody is assigned yet. That is correct: automatic assignment is off (Step 2.7), so it waits in the admin's queue.

If the payment fails or you close the sheet, that is worth seeing too, and costs nothing:

What you do	Expect
Close the Razorpay sheet without paying	"Payment was not completed. You can try again below." The button now reads Pay ₹1,999 Now , not Request Booking. The appointment already exists, unpaid.
Fail three times	An amber escape hatch appears: "Having trouble paying? Your booking is saved as pending..." with a Go to Dashboard link.
Tap Pay ₹1,999 Now again	Checkout re-opens against the same appointment. No second appointment, no second order.
Press Back from Step 3 to Step 2	The draft is abandoned; the button returns to Request Booking . The unpaid appointment stays in the dashboard.
Reload the page mid-wizard	The wizard restarts at Step 1. No wizard state is kept. Any appointment already created is still in the dashboard as unpaid.

Step 4.5 - Read the patient's Overview

Do this. On `/patient/dashboard`, read the screen top to bottom.

Expect

- A strip of **four figures**, then the activity feed, then quick actions - in that order.
- The sidebar shows **Overview, Book a Session, Sessions, Payments, Health Profile, Edit Profile. Programmes is absent** - the patient has not bought one, and a screen that can only ever be empty is not in the sidebar. **Back to Home** sits at the foot of the nav.
- Every time on this screen is in **India Standard Time**, whatever your laptop is set to. A session booked for 4 PM must not read `10:30 AM` anywhere - that is the clinic-time rule broken, and it is a **P1** because two people reading one screen then disagree about when the session is.
- The feed carries the booking. Nothing claims the patient did something they did not do.

Then check the loading states. In DevTools → **Network**, set throttling to **Slow 3G** and move through every sidebar entry.

Expect. Each screen shows a skeleton with **its own label** - `Loading your dashboard`, `Loading booking`, `Loading your sessions`, `Loading your payments`, `Loading your health profile`, `Loading your profile` - and **the sidebar stays on screen throughout**. A screen that blanks the chrome while it loads is a **P2**; one that flashes white, or shows another screen's label, is the same.

Turn throttling off.

Step 4.6 - Fill in the patient's own profile

Do this. Open **Edit Profile** and fill in every field:

Field	Value
Date of birth	<code>1990-04-12</code>

Field	Value
Gender	Female
Address line 1	12, 3rd Cross, Indiranagar
Address line 2	Near Metro Station
City	Bengaluru
State	Karnataka
PIN code	560038
Emergency contact name	QA Contact A
Emergency contact phone	+91 98765 43299

Save, then reload the page.

Expect. Everything survives the reload. A patient's own details save immediately - there is no review step for these, unlike a therapist's credentials.

Then upload a report. Still on the patient's screens, open **Health Profile** and use the reports uploader. You need a small PDF; any file will do, named something you will recognise:

File	Type to pick
Spine_Report_E2E.pdf	Scan or X-ray

Expect. It uploads and is listed. **The uploader is open even though the health profile itself is locked** - the therapist has not filled the first record yet, and this is the one useful thing a patient can do beforehand.

Try two refusals while you are here:

File	Expect
Any file over 10 MB	Refused with a size message.
A <code>.txt</code> file	The picker will not accept it.

Step 4.7 - Note what the patient cannot do yet

This is the state Part 6 changes, and it is worth reading now so the change is visible later.

Do this. On **Health Profile**, read the screen.

Expect

- There is **no** "start the questionnaire" call to action and **no** answered counter (`0 of 7`). Both are **absent**, not greyed out - the record is not the patient's to start.
- The Overview's health cell reads ``-`` on slate, **not `0%` on amber**. Amber would be asking the patient for something they cannot give.
- There is no amber banner telling them to complete anything.
- The reports uploader still works.

Why. A patient's health record does not exist until a therapist has triaged them and written the first entry. Locking the patient out until then is deliberate: the alternative is a patient filling in a form for a condition type nobody has decided yet.

Step 4.8 - Register Patient B, the other way

You need a second patient for the isolation checks, and registering them through `/patient/register` proves the other rule: a standalone signup **always** waits for a human.

Do this

1. Sign out. Open `/patient/login` and tap **Register Account**.
2. Fill it in:

Field	Value
Full Name	QA Patient B
Email Address	qa.patient.b@example.test
Phone	+91 98765 43211
Password	QaTest!2024pass
Confirm Password	QaTest!2024pass
Referral Code	<i>leave blank</i>

1. Tap **Create Account**.

Expect

- You land on `/pending-approval` - not the dashboard, and **not** any "check your email" screen. A screen telling this patient to check their inbox is a **P1**: email confirmation is off in this product, so nothing will arrive and they will wait for ever.
- Type `/patient/dashboard` into the address bar. You are redirected back to `/pending-approval`.

Then prove the gate is in the route, not only the screen. Still signed in as Patient B, DevTools → Console:

```
const r = await fetch("/api/appointments/create", {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify({ slotTime: "2026-12-01T10:30:00.000Z" }),
});
({ status: r.status, body: await r.text() });
```

Expect: ``200``, and an unpaid, unassigned session appears. **That is correct and is not a defect.** This route gates on the account being *active*, not approved - an unapproved self-signup patient has to be able to hold the row they are about to pay for, and an unpaid row grants nothing. What must be refused is a **suspended** account, which you check at Step 12.2.

Why ``10:30:00.000Z`` and not ``10:00:00.000Z``. A slot must start on the hour in the booking's own **timezone**, and with no timezone in the body that is India. `10:00Z` is 15:30 IST and is correctly refused

with Sessions start on the hour. Pick a time like 6:00 or 7:00. 10:30Z is 4 PM IST. If you see that refusal, you have found the rule working, not a bug.

Delete the stray session later from **Sessions → All Sessions**, or leave it - Step 5.2 uses it.

Step 4.9 - Approve Patient B, as the admin

Do this. In the admin browser, open **Today → Approvals** and approve QA Patient B.

Expect. They disappear from the queue, the badge falls by one, and signing in as Patient B now reaches /patient/dashboard.

Then decline something, to see the other outcome. Register a throwaway patient (QA Patient Z , qa.patient.z@example.test , phone +91 98765 43219 , same password) and **decline** it from the same queue with the reason:

Duplicate registration, patient already has an account.

Expect. The account stays unapproved and cannot reach a dashboard. It is **not** deleted - declining is not deleting, and the row is still in People → Patients marked accordingly.

Step 4.10 - Checkpoint

	Should be
Patient A	Approved, one paid session tomorrow at 4 PM, unassigned, profile filled, one report uploaded
Patient B	Approved, possibly one stray unpaid session from the console call
Patient Z	Registered and declined
Health profile	Locked for Patient A - no CTA, no counter, - on the Overview

6. Part 5 - Assign, join and deliver the session

What this part does. Takes Patient A's paid session from "nobody is assigned" to "delivered and written up". It is the spine of the clinic's operational day, and every money figure in Part 10 comes from it.

Time. About 45 minutes.

Step 5.1 - Find the session in the admin's queue

Who you are. The Master Admin.

Do this

1. Open **Today**.
2. Read the "needs a person" figure and the queue list beneath it.
3. Tap the figure.

Expect

- The figure and the list **agree**. A strip reading 23 over a list of four is the exact defect this check exists for - **P1**.
- Tapping it opens **Sessions → All Sessions** already filtered to the rows it counted, **not** the whole table. Filtering you have to redo by hand is the failure; so is a count that opens an unfiltered list and therefore looks wrong.
- Patient A's session is there: paid, tomorrow at 4 PM, **no therapist**.
- The row carries a chip reading **Tap to assign** - not "Reschedule / Reassign". A session nobody has ever been assigned to must not describe the action as editing something that already happened.

Then tap somewhere else and come back. Move to another screen and return to All Sessions.

Expect. The preset is **gone** - it is one-shot, so a filter never becomes something an admin cannot find the source of. Tapping the Today figure again re-applies it.

Step 5.2 - Assign Therapist A

Do this

1. Tap Patient A's session row. The detail drawer opens.
2. Read what the drawer leads with.
3. Assign **QA Therapist A**.

Expect

- The drawer **leads with the assign control**, with the reschedule form kept below for when the time has to move too.
- The session becomes **confirmed**.
- A **Meet link** appears on it, and on the patient's own session card.
- The therapist's dashboard now lists the session.

If there is no Meet link, look at **Settings → System Health** before reporting anything:

What System Health says	What it means
Google shows Not set up	Nobody wired Google up in this environment. Not a defect - it is a state, not a fault, and the run continues without video links. Mark the Meet checks N/A.
Google shows Needs you now with a dead-credential message	One refresh token has died, so every session fails identically. The card states the length, an eight-character fingerprint and whether the stored value has stray whitespace - that is how you tell "the permission died" from "the server is still holding the old value".
The session sits in Session Links with an error	The sweep will retry it, capped. A manual Retry resets the counter.

One thing to watch for and report. Tap **Retry** on a **home visit** (you will have one after Part 8). A home visit **never** has a Meet link by design - there is nothing to join. If Retry answers `502 Retry failed`, or if every click creates a **new calendar event**, that is a **P0**: three duplicate invites once reached one patient that way.

Step 5.3 - Check the session reads the same on every screen

The same row is rendered by four different people. They must agree.

Do this. Open, in turn, and compare the date, the time and the status:

Screen	Where
The patient's Sessions	<code>/patient/dashboard/sessions</code>
The therapist's Sessions	<code>/therapist/dashboard/sessions</code>
The admin's All Sessions	Sessions → All Sessions
The admin's Schedule	Sessions → Schedule (calendar)

Expect. One date, one time, one status, everywhere - and the time is **4 PM**, in India Standard Time, on all four. A screen showing `10:30 AM` for the same row is the clinic-time rule broken.

Also check the patient's Sessions screen is one list. Upcoming / Past / Cancelled filters, not separate sidebar entries for video and home visits. The Video / Home visit filter appears only once this patient has both, which they do not yet.

Step 5.4 - Check the join window

Do this

1. As the patient, look at the session card now (it is tomorrow).
2. As the therapist, look at the same session.
3. Open **Settings → Booking Rules** as the admin and read the **join window** and the **Session Completed cutoff**.

Expect

- Well before the slot, the join control is **not** live - it names when it opens rather than being a dead button.
- Inside the window, **Tap to Join** works for both parties.
- Past the cutoff - the admin-set number of minutes after the slot time - every join control reads **Session Completed** instead, **including the admin's own**. A session an hour past its start must read the same way on every screen it appears on.

If you cannot wait for real time to pass, use the Debug bar's simulated time rather than editing the database. Changing a session's slot to the past by hand also changes what the money screens count.

Step 5.5 - Try to complete it too early, then properly

Who you are. QA Therapist A.

Do this

1. **Before** the join window opens, try to mark the session complete.
2. Then, inside or after the window, mark it complete.

Expect

- Step 1 is **refused**. Completing a session is a financial write with a clinical name: `completed` + `paid` is the exact and only condition that makes the therapist's revenue share payable, so a session cannot be closed before the window in which it could have been started. A therapist who can mark tomorrow's session done today and be owed for it is a **P0**.
- Step 2 succeeds. The session reads **Completed** on all four screens from Step 5.3.

Then check the two refusals that protect the money, each on a session you set up for it:

Situation	Expect
A session with no payment behind it	The therapist cannot complete it.
A cash home visit with no cash recorded	The therapist cannot complete it - collect first, which is the right order anyway.

An **admin** can complete a session in either of those states, deliberately: a backfill or a correction is exactly what the override lane is for. What an admin cannot do is complete one from a desk that only reads Sessions - Step 12.3 checks that.

Step 5.6 - Write the session note

Who you are. QA Therapist A.

Do this

1. From the completed session's card, open the session note dialog.
2. Fill all four fields:

Field	Value
What was treated	<code>Lumbar assessment. Reduced flexion, pain on end-range. PA mobilisations L4-L5.</code>
How the patient responded	<code>Good tolerance, reported easing during the session. No radiating symptoms today.</code>
Home exercise	<code>Cat-cow x10, twice daily. Walking 15 minutes after lunch.</code>
Plan for next time	<code>Reassess flexion range. Add glute bridge progression if pain stays under 4.</code>

1. Save.

Expect

- Saved, and the therapist's Overview **Notes to write** figure falls by one. At zero it reads `Every delivered session is written up`.
- The note is editable for **24 hours**, and every edit inside that window keeps a copy of what it replaced.
- **The patient cannot see it anywhere.** Check the patient's Health Profile and their export - session notes are clinician-only and are excluded from both on purpose. A patient who can read a session note is a **P0**.

Then check the contact scanner on this field. Open the note again and try to save each of these:

What you type	Expect
Grade III PA mobilisation x3 sets, 30s hold. 10 reps, 2x daily. Order ref 90210.	Saves normally. Clinical text full of numbers must not fire the scanner - a check that cries wolf is a check nobody reads.
Call me on 9876543210 before the session	Saves, and is recorded. A phone number is flagged, not blocked. It appears on the admin's flagged-messages panel.
Pay me directly on 9876543210@okhdfc, it's cheaper	Refused. A payment handle is blocked outright.

Restore the real note text afterwards.

Then edit the note and check the previous version was kept. Change one sentence and save again.

Expect. The edit lands, and what it replaced is **kept** rather than overwritten. Notes stay editable for **24 hours**; past that the submit route refuses the edit, whatever the screen offers. A note that can be silently rewritten days later is a clinical record with no history - **P1**.

Step 5.7 - Check the patient's phone is masked

Who you are. QA Therapist A.

Do this

1. Open **My Patients** and then Patient A.
2. Read the contact details.
3. Inside the session's join window, use **reveal contact**.

Expect

- The phone is **masked** and the email is **not shown at all** - it is not loaded onto these screens in the first place.
- Revealing works inside a video session's join window, and on a home visit's own day.
- It is **refused** outside that window, and refused for a cancelled session.
- Every reveal is recorded. As the admin, check the reveal log has a row. **A reveal that could not be recorded is refused** - unlike the audit log, this one is not best-effort, because a reveal with no trace is the one outcome the route must not produce.

Step 5.8 - Rate the session

Do this. As Patient A, rate the completed session **4 stars** with the comment:

Clear explanation and a plan I can actually follow at home.

Expect. The rating saves. The therapist's Overview header now reads **Your Rating: 4.0 (1 rating)** instead of **No ratings yet**.

Then, as the admin, hide that therapist's rating from public pages and check **/team** no longer quotes it, and that the therapist's own header gains **- hidden from public pages**. Put it back.

Step 5.9 - Reopen a completed session

Closing a session is what makes a therapist's share payable, so undoing it has to undo the whole of it.

Do this

1. As the admin, open the session Therapist A completed at Step 5.5 and reopen it.
2. Read what the screen says will happen before confirming.
3. Look at the session afterwards.
4. Look at both ratings.

Expect

- The session returns to **confirmed**, and the record of *when* it was completed is **cleared** with it. A row reading `confirmed` while still carrying a completion time is a contradiction, and it is exactly the evidence a risk detector should no longer be looking at - **P1**.
- **Both sides' ratings are destroyed**, and you were told so before confirming. The 4-star rating from Step 5.8 is gone.
- The therapist's **Owed to you** figure falls back by that session's share.

Then have two admins reopen it at once, in two browsers, and tap within a moment of each other.

Expect. Exactly **one** of them does it. The second is refused or is a no-op rather than reopening an already-reopened session and destroying a rating somebody has since left again.

Re-complete the session and re-rate it before moving on - Part 10 counts it.

Step 5.10 - Checkpoint

	Should be
Patient A's session	Completed and paid, delivered by Therapist A
Session note	Written, clinician-only
Rating	4 stars, showing on the therapist's header
Flagged message	One recorded, from Step 5.6
Contact reveal	One logged

7. Part 6 - The health record

What this part does. The therapist triages Patient A, writes the first health record, and records a physical examination. That first fill is what unlocks the patient's own access - the screen you read at Step 4.7 changes here.

Who you are. QA Therapist A, then Patient A.

Time. About 45 minutes.

Step 6.1 - Triage the patient

Do this

1. As QA Therapist A, open **My Patients** and then **QA Patient A**.
2. Open the triage dialog.
3. Answer the four questions:

Question	Answer
How old is the patient?	18 to 64
What brought them in?	Injury, strain or overuse
Any of these present?	None of these
Any concern about milestones...?	not shown - it only appears when the age is `Under 18`

1. Read what the dialog suggests before accepting anything.

Expect

- The dialog shows **everything at once**, with headings. That is deliberate: a clinician filling this after every assignment wants to scan it, where the patient's own questionnaire is paced one question at a time.
- It **suggests** **Orthopaedic**, **with its reason shown**, and does **not** accept it for you. A suggestion applied automatically is a **P1** - the clinician confirms.
- The word on screen is **condition type**, never "specialty" and never "case".
- Spelling is British - **Orthopaedic**, **Paediatric**. An American spelling in one label reads as a typo beside the others.

1. Confirm **Orthopaedic**.

Step 6.2 - Write the first health record

The orthopaedic set is **seven questions**. Fill every one.

Do this. Still on Patient A, fill the record:

Question, as shown	Answer to enter
What's the main issue you'd like help with?	Lower back pain that spreads into my right hip
How long has this been going on?	About four months
Overall severity right now (0-10)	6
Where does it hurt? (tap each area, rate 0-10)	Tap Lower back → rate 7 ; tap Right hip → rate 5
What makes it worse?	Sitting for more than an hour, and bending to pick things up
What helps or relieves it?	Walking, and lying flat for ten minutes
Anything else the therapist should know?	I work at a desk nine hours a day. No previous surgery.

Save.

Expect

- It goes **live immediately**. There is no approval queue in front of a clinician writing the first record - the patient is locked out of their own health profile until it lands, so a queue here would leave them on a read-only screen after a session that has just ended.
- It is still **recorded**: open the patient's Review History and confirm an already-approved entry is there. Live is not unrecorded.
- The copy addresses **the clinician**, not the patient. A clinician being told "this is your own account of your condition, in your words" is the shared-copy failure this rule exists for - report it as a **P2**.

Then submit the identical thing again, without changing a single answer.

Expect. Nothing is written - no record update, no second history entry. An identical resubmission is a no-op, keyed on whether anything clinical actually changed. Ten taps producing ten history entries is the defect; two is the half-fixed version of it. Check Review History still has exactly one entry from this step.

Step 6.3 - Record a physical examination

The Pain Map is **orthopaedic** and stays one. It is the therapist's own clinical layer, separate from the record above.

Do this

1. On Patient A's chart, find the Pain Map.
2. **Tap the body figure** on the lower back region - do not look for a dropdown.
3. In the exam dialog, fill the grouped questions with whatever clinical values the form offers, keeping the region in the header visible while you type.
4. Save.
5. Record a **second** assessment on the same region a moment later, with a lower pain value.

Expect

- The region is chosen by **tapping the figure** (or a chip in the dialog), never a `<select>`, and it **stays in the dialog header** while the clinician types.
- Questions are **grouped**, not listed flat.
- It posts **live**, with no review step.
- The second assessment is a **new row**, never an edit of the first - so the screen can show a trend against the previous visit. If the first disappears, that is a **P0**: this layer is append-only.
- The figure is an anatomical silhouette with **front and back as separate SVGs** that stack on a phone, rather than one figure shrunk until the tap targets are smaller than a fingertip.

Check the scale. Every exam figure the app prints is out of **ten**, matching how the patient rates their own pain. A strip reading *"How you rate it 6/10"* beside *"Last exam found 34%"* is two different measurements on one screen - **P2**.

Then check the gate sits beside the thing it gates. The "request access to edit" card is **inside** the Pain Map card, stating what is readable regardless and what needs approval - not three sections further up the page.

Step 6.4 - Watch the patient's screen unlock

Who you are. QA Patient A.

Do this. Open **Health Profile** and compare it with what you read at Step 4.7.

Expect

- The record is there, showing **answers, not inputs**.
 - The Overview's health cell now shows a real percentage rather than **-**.
 - Nothing claims the patient answered questions a clinician wrote. Look at the counter: it must not read **3 of 7 answered** over "Add the missing answers" for answers the patient never gave. **Attribution is not a nicety on a medical record** - a counter that credits the patient with the clinician's typing is a **P1**.
 - The word on every patient-facing screen is **Health Profile**. Not "Patient Care Intake", not "condition data", not "the questionnaire", and never "chart" - that is clinician register.
 - The patient is never shown a category word. Their care reads as **Orthopaedic physiotherapy**, never **ortho**, and the words "triage" and "onboarding" appear nowhere on their screens.
-

Step 6.5 - Export the record

Do this. As Patient A, use the export on Health Profile.

Expect

- A **PDF** downloads, named **QA Patient A_PT####.pdf** using that patient's own code.
 - It is typeset and readable - the thing a patient does with an export is hand it to another clinician.
 - **Session notes are not in it**. Neither are they in the JSON form. Check.
 - The uploaded report from Step 4.6 is listed as metadata.
-

Step 6.6 - Check one patient cannot read another's record

Do this. Sign in as **QA Patient B** and try to reach Patient A's health profile - by URL if the interface gives you no link.

Expect. Refused. Nothing of Patient A's renders, and no error message names a table or a column.

Then as QA Therapist B - who is not assigned to Patient A - open **My Patients**.

Expect. Patient A is **not** listed. Therapist B has no route to that record. Any path by which one patient's clinical data reaches another patient, or an unassigned clinician, is a **P0** and stops the run.

Step 6.7 - Try to edit the record on the patient's behalf

There is a line between **creating** a record and **editing** one, and it is worth seeing.

Do this. As QA Therapist A, try to change an answer in Patient A's live record.

Expect. You are asked to **request access**, which an admin approves - unlike the first fill, which needed only assignment. Deciding what kind of patient this is, and writing down what they told you in a session you ran, is the clinician's own record; editing a live record on the patient's behalf is editing their own account of their history.

Then, as the admin, approve that request from **People → Condition access**, and confirm the therapist can now submit an edit that goes through review.

Step 6.8 - Checkpoint

	Should be
Patient A	Triaged Orthopaedic , seven answers on file, health profile unlocked
Pain Map	Two assessments on one region, both kept
Export	PDF, named after the patient, no session notes
Isolation	Patient B and Therapist B can reach none of it

8. Part 7 - The recommendation, and buying a programme

What this part does. The only route by which a patient buys more than one session: the therapist recommends a programme, the clinic approves it, the patient pays, and the whole run of sessions goes into the diary.

Time. About 70 minutes.

The rule this part exists to prove. A therapist picks a **package**, never a price. There is no price field, no session-count field and no discount field on a recommendation - not hidden, not disabled, *absent*. If you find one, that is a **P0** and worth stopping for.

Step 7.1 - Write the recommendation

Who you are. QA Therapist A.

Do this

1. Open the completed session from Part 5 and open its note dialog.
2. Find the recommendation panel.
3. Answer the two questions that decide everything:

Question	Answer
Which condition?	QA Back & Spine Care
How many sessions?	the option that resolves to `QA Spine Recovery 6 Sessions`

1. Fill the four clinical fields:

Field	Value
Hands-on required	Yes
Sessions per week	2
Why this, for this patient	Persistent mechanical low back pain, four months, desk-driven. Needs a supervised progression rather than single sessions.
Anything they should do or know	Keep walking daily. Stop any exercise that sends pain below the knee and tell me at the next session.

1. Submit.

Expect

- The programme list offers **only** programmes filed under **QA Back & Spine Care** - P1 and P3. **QA Neuro Rehab 8 Sessions** is **not** on offer, because it belongs to a different condition. A programme from somebody else's condition appearing here is a **P1**.
- There is nowhere to type a price, a session count or a discount.
- It saves as **waiting for the clinic** - not live.

Then check what the patient sees. Sign in as Patient A.

Expect: nothing. Not a greyed-out card, not "your therapist has recommended something, pending approval" - **nothing at all**. The recommendation is absent from their screens entirely until the clinic approves it. A visible pending recommendation is a **P1**, and a *purchasable* one is a **P0**.

Step 7.2 - Try to buy it anyway

Do this. Still as Patient A, DevTools → Console:

```
const r = await fetch("/api/care-plan/create-order", {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify({ carePlanId: "<paste the care plan id if you can find it, or any uuid>" }),
});
({ status: r.status, body: await r.text() });
```

Expect. Refused. Hiding a card is presentation; refusing the order is the rule. A **200** here is a **P0**.

Step 7.3 - Read the review queue

Who you are. The Master Admin.

Do this

1. Open **Today**. Read the inbox row for recommendations.
2. Open **Sessions → Recommendations**.

Expect

- The queue is **oldest first**, and each card is aged **in words** - **12 minutes ago** - not stamped with a date. This is a work queue, not a record.
 - Today's inbox row is urgent **only** once something has waited past about four hours - not merely because the queue is non-empty. A badge that is always on is a badge nobody reads, and this is the one queue with a patient waiting behind it.
 - Each card states **how many sessions or visits that patient already has unused**. Patient A has none. That figure is the commonest reason to turn a recommendation down, and it used to be invisible without leaving the queue.
 - The card names the therapist whose judgement it is.
-

Step 7.4 - Turn it down, then approve the rewrite

Both outcomes are worth seeing, and the rejection is the one that has to reach the therapist.

Do this

1. **Turn it down** with the reason:

Six sessions is more than this presentation needs. Please propose a shorter block.

1. Sign in as QA Therapist A and look for it.
2. Write a **new** recommendation, this time choosing `QA Single Session` (P3), with the same four clinical fields.
3. As the admin, **approve** it in one tap.

Expect

- A reason is **required** to turn one down - the therapist has to act on it, and "Not approved" alone says the recommendation is gone without saying what to write instead. Try a reason under ten characters: refused.
- The rejection reaches the therapist **twice**: as an item on their dashboard carrying the reason, **and** on the patient's chart beside the thread. The feed scrolls away; the chart is where a clinician goes to rewrite.
- **Approving needs no reason at all.** One tap. Tapping an approval with a sentence meaning "fine" is how a reason column fills up with "ok" and stops being worth reading - and it makes the patient wait longer for something nobody objected to. If the approve button demands a reason, that is a **P2** in the other direction.
- Every decision is recorded. Check **Logs → All Activity**: there are rows for the rejection and the approval, each naming who and when.

Step 7.5 - Approve with different numbers

Worth doing once, because it is deliberately **not** an edit.

Do this. Have the therapist submit one more recommendation (P1, six sessions). As the admin, use **approve with changes** to publish P3 instead, with the reason:

Starting with a single review session before committing to a block.

Expect

- A **new version** is written, attributed to the **therapist** as its author and recording the **admin** as the person who typed it. Both names appear.
- The therapist's original is **still in the thread**, marked superseded. Rewriting a clinician's version under their name would be a lie about who decided what - if the original is gone, that is a **P0**.
- A reason is required here, because this takes something away from somebody.

Step 7.6 - Check a stale offer is caught before the patient meets it

Do this

1. Have the therapist submit a recommendation for **P1**.

2. Before approving it, go to **Catalog → Packages** and change P1's price to `12000`.
3. Now try to approve the recommendation.

Expect. The approval is **blocked**, with a sentence naming the drift - the session count and the price are the two figures a patient reads and pays, and they no longer match. The alternative is that the patient discovers the clinic's stale data by having their payment refused at the last step of checkout.

Then try to reject the same one. It is **allowed**. Refusing to let an admin close a thread because its package moved would trap exactly the recommendation that most needs closing.

Put P1's price back to `9999`.

Step 7.7 - Publish the real one and let the patient buy it

Do this

1. As the therapist, submit a recommendation for ``QA Spine Recovery 6 Sessions`` (P1) with the four clinical fields from Step 7.1.
2. As the admin, approve it.
3. Note the moment you approved.

Expect. The offer window is stamped **at approval**, not at authoring. A plan that sat in the queue for two days must not reach the patient with two days already spent off its clock. Check the expiry on the patient's card against the approval time, not the writing time.

Now, as Patient A:

1. Open **Suggested Sessions**.
2. Read the offer card.
3. Pay for it with the same test card (`4111 1111 1111 1111`), any future expiry, any CVV, **Success**).

Expect

- The card quotes **₹9,999** and **6 sessions**, the figures from the admin's catalogue - not anything the therapist typed.
 - After paying, the screen lands on a **confirmation and one next step** - what arrived, what they own, then the scheduler. **A blank screen here is a P1**: this is the highest-intent moment in the product, and it used to go blank because the offer card had been accepted and nothing replaced it.
 - "I'll do it later" is a real option, not a trap.
-

Step 7.8 - Schedule the run

Do this. On the scheduler that follows the payment, read what it opens with before changing anything.

Expect

- The calendar opens **already answered** - a whole run of dates proposed from the clinician's own cadence (two a week, minimum 24 hours apart, maximum three a week, inside the 90-day validity).
- A day that cannot take the run's hour is **skipped**, not substituted. Somebody who asked for five o'clock and was handed nine in the evening because it was the only slot clearing the lead time has been given a schedule they did not ask for.
- It **stops at the validity** - it proposes fewer than six rather than proposing sessions the patient would lose.

Now book them. Accept the proposal, or adjust a date and book.

Expect

- Every session is booked, **auto-assigned to Therapist A and auto-confirmed** - the first therapist on a programme locks it, and every later session goes to them without passing through the admin's queue.
- Each gets its own Meet link.
- The balance falls to zero as you book them.

Then check the two batch rules. Try to book two sessions **12 hours apart**, and try to book **four in one week**.

Expect. Both refused, with readable messages - P1's minimum gap is 24 hours and its maximum is three a week. Those are the catalogue's own rules reaching the booking.

Step 7.9 - Leave one unbooked and check the dashboard keeps asking

Do this. Cancel one of the booked programme sessions so the balance goes back to one, then open the patient's **Overview**.

Expect. An item that stays at the **top** of the feed until the balance is spent - it is waiting on the viewer, so it is pinned above dated items rather than sinking as it ages. A programme paid for a month ago with a session unbooked is exactly the item that must not drift down the list.

And check the balance agrees everywhere:

Screen	Should say
The patient's programme widget	6 sessions, 1 unused
The therapist's programme list	the same
Admin → Catalog → Purchases	the same
The scheduler	the same

A figure that disagrees between two of these is a **P1**.

Step 7.10 - Check a purchase cannot be rewritten by the catalogue

Do this

1. As the admin, change **P1** to **4** sessions at **₹12,000**.
2. Open Patient A's programme widget, and the admin's Purchases row.

Expect. The purchase still reads **6 sessions at ₹9,999** - what was actually bought. A purchase that follows the live catalogue is a **P0**: an admin re-pricing a package would silently rewrite what somebody already owns.

Put P1 back to **6** sessions at **₹9,999**.

Step 7.11 - Withdraw one, and find what cannot be withdrawn

Do this

1. Have the therapist submit another recommendation and leave it queued.
2. As the admin, **withdraw** it with the reason `Therapist is on leave and cannot revise this.`
3. Then try to withdraw **the purchased** plan from Step 7.7.

Expect

- The queued one closes, freeing the patient's one-open-plan slot. Refusing here would leave the queue holding a thread nobody intends to approve while the patient's slot stayed taken.
- The **purchased** one cannot be withdrawn at all. The patient has paid and the sessions exist; the honest lane is a refund or a credit adjustment, each with its own screen.

Step 7.12 - Write one on a therapist's behalf

Do this

1. As the admin, put **QA Therapist A** on leave.
2. Open **Sessions → Recommendations** and find the panel for writing one on their behalf.
3. Write a recommendation against one of Therapist A's own completed sessions, with the reason `Therapist on leave, patient waiting on a plan.`

Expect

- The panel is **there** even when there is no session to write against or no recommendable programme - it says which of the two is missing rather than being simply absent. An admin opens this screen because a patient is waiting, and a missing panel reads as a feature that does not exist.
- The programmes offered are narrowed to **the chosen session's own condition**, and changing the session **drops the draft** so a package for somebody else's condition cannot be carried across.
- Whose name it goes out in is stated **at the button**, not in a subtitle two screens up.
- The saved version names the **therapist** as author and the **admin** as the person who entered it.
- There is still no price field.

Take Therapist A off leave.

Step 7.13 - Checkpoint

	Should be
Patient A	One purchased programme, 6 sessions, 1 still unbooked
Sessions	Five more in the diary, all with Therapist A, all confirmed
Recommendations	One rejected, one approved-with-changes, one withdrawn, one purchased
Logs	A row for every one of those decisions

9. Part 8 - The home visit

What this part does. Buys and delivers a visit at the patient's address, both ways of paying for it. The travel fee is the thing to watch throughout: it is a reimbursement paid to the therapist in full, never

revenue, and never discounted.

Time. About 50 minutes.

Step 8.1 - Book a visit, paying online

Who you are. QA Patient A.

Do this

1. Open `/book-home-visit`. Confirm the header reads **Step 1 of 4**.
2. Tap **Pincode**, enter `560038`, tap **Check**.
3. Read the line that comes back.
4. Fill the address:

Field	Value
Address line 1	<code>12, 3rd Cross, Indiranagar</code>
Address line 2	<code>Near Metro Station</code>
City	<code>Bengaluru</code>
State	<code>Karnataka</code>
PIN	<code>560038</code>

1. **Continue.** On **When suits you?**, pick a date **at least 24 hours out** and an arrival time. Read the notice about notice.
2. **Continue.** On **About you**, choose `QA Home Visit - Single - 1 visit`.
3. **Continue.** On **Review and pay**, read every line of the breakdown before tapping anything.
4. Choose **Pay online** and pay with `4111 1111 1111 1111`, **Success**.

Expect

- Step 3: a teal line reading *"Yes - we visit Indiranagar, Bengaluru. Travel to this area is ₹150 per visit."*
- Step 5: the copy says home visits need at least **24 hours'** notice - deliberately longer than the online session's 12, and read from its own setting. If changing the online lead time at Step 12.1 also changes this one, that is a **P1**: the two are independent.
- Step 7: the breakdown shows **three** figures - programme `₹2,499`, travel `₹150`, total `₹2,649`.
- **The button charges exactly the total shown.** ₹2,649, not ₹2,499. Quoting one figure and charging another is a **P0**, and this is the place it has happened before: a four-visit programme in a ₹150 area was ₹600 out because the card printed the programme price alone.
- After paying: a confirmation. The visit appears on the patient's Sessions screen, and the **Video / Home visit** filter now appears there - they have both kinds.

Then check the address was snapshotted. Go to **Edit Profile** and change the saved address to something else. Re-open the booked visit.

Expect. The visit still carries the **old** address. It was copied onto the appointment at purchase, not referenced live - editing a saved address must never rewrite a visit already booked or delivered. Put the address back.

Step 8.2 - Check the unserviceable path

Do this

1. Open `/book-home-visit` again and enter `560025`. Tap **Check**.
2. Leave a waitlist entry: `QA Patient B`, `+91 98765 43211`, then **Tell me when you do**.

Expect

- An amber panel: *"We don't visit 560025 yet."* and *"Leave your number and we'll tell you the moment we do. Nothing has been charged."*
- **No address form and no package picker appear.** Serviceability is checked before an address is even collected.
- The panel offers a link to an **online consultation**, which is available anywhere.
- After submitting: *"Thanks - we'll be in touch."*
- As the admin, **Catalog → Service Areas** shows the waitlist entry and its badge count has risen. Change its status and the badge clears.

And the four bad pincodes:

Value	Expect
<code>56003</code>	<code>Enter a valid 6-digit pincode.</code>
<code>0560038</code>	<code>Enter a valid 6-digit pincode.</code>
<code>abcdef</code>	<code>Enter a valid 6-digit pincode.</code>
<i>(blank)</i>	Nothing harmful happens; a validation message appears

Step 8.3 - Prove a four-visit package cannot be bought

Do this

1. Look at `/home-visit` and at the patient dashboard's booking screen.
2. Then, as Patient A, DevTools → Console:

```
const r = await fetch("/api/home-visit/create-order", {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify({ packageId: "<paste HV2's id from the admin's catalogue>", pincode: "560038"
}),
});
({ status: r.status, body: await r.text() });
```

Expect

- **HV2 is not offered** on either screen. Only HV1, the single visit, has a Book button. HV2 on the booking screen is a **P0**.
- The console call is **refused**. Two or more visits is a programme, and a programme comes from a recommendation - hiding the card is not the rule, refusing the route is.

And check the same for a stale link. Open `/book?package=<any package id>`.

Expect. Not silence, and not a different amount of money - a panel reading *"Programmes come from your therapist now"* with an explanation and a **Book a first session** link. Taking a different amount than

somebody came for is the one outcome a removed checkout must not produce.

Step 8.4 - Book a second visit, paying cash at the door

Do this. Repeat Step 8.1, but at **Review and pay** choose **Pay at the visit**.

Expect

- The purchase is created and the visit is confirmed, with `payment_status` still reading unpaid for its whole life. **That is correct for cash** - never read a home-visit purchase's payment status the way you would an online one; check how it is being paid first.
- The patient's screens do not claim it is paid.

Then switch cash off (Settings → Programmes & Home Visits → allow cash on visit) and try again.

Expect. The cash option is gone from Step 4, and the route refuses it if called directly. Switch it back on.

Step 8.5 - Deliver the cash visit and record the money

Who you are. QA Therapist A - assign yourself the visit as the admin first if it is unassigned.

Do this

1. Try to mark the cash visit **complete** before recording any cash.
2. Record the cash collection.
3. Then complete it.

Expect

- Step 1 is **refused** - a cash visit collects first, which is the right order anyway.
- Step 2 takes **no amount from you**. The therapist asserts that money changed hands; the system owns the number, reconstructed from the purchase. If this screen has a field where the person holding the cash types how much the clinic knows about, that is a **PO** - it is a one-field withdrawal.
- Step 3 succeeds.

Then, as the admin, correct the amount. Open **Money → Cash Ledger**, find the collection, and correct it with the reason:

Patient was short at the door, agreed the balance next visit.

Expect

- The correction needs a reason and is recorded in **Logs → All Activity**.
 - A correction on a visit whose cash has **already been remitted** is refused - that transfer has gone out, so the fix is an adjustment against the next payout rather than a silent edit of a settled one.
-

Step 8.6 - Check travel is paid to the therapist, not kept as revenue

Do this. As the admin, open **Money → Summary**, then **Money → Payouts**.

Expect

- The **₹150** travel on the online-paid visit is part of what **Therapist A is owed**, in full.

- It is **not** in revenue. If the clinic's share moved by ₹150 when that visit was delivered, the travel bill has been folded into revenue - **P0**, and it means the therapist is funding their own transport.
- Therapist A's home-visit share is **65%** (Step 3.5). Check the payout maths uses that rate, not their 60% online rate.

Then deliver one with Therapist B, whose home-visit share you deliberately left unset.

Expect. Their **55%** ordinary rate applies - not zero, and not Therapist A's 65%.

Step 8.7 - Check the travel buffer

Do this

1. As the admin, read the **travel buffer** on Settings → Programmes & Home Visits (default 45 minutes).
2. Book Therapist A a home visit, then try to book them a second visit starting **30 minutes** after the first ends.

Expect. Refused as a clash - the conflict check is padded by the buffer on **both** sides of the new visit, because a therapist finishing one visit cannot be at another minutes later. An online session passes **0**, so the same 30-minute gap between two video sessions is allowed.

Step 8.8 - Cancel a visit and check its own refund window

Do this

1. Read the **home-visit cancellation refund window** on Settings → Programmes & Home Visits.
2. Cancel a paid home visit **outside** that window.
3. Cancel another **inside** it.

Expect

- Outside: refunded, and the refund **includes the travel** that was charged. Refunding the service line alone leaves the patient paying for a journey nobody made - check the figure.
- Inside: not refunded, and the card says **why**, naming the window that actually applied - the home visit's own, not the online 24 hours.
- Every refund states a reason, even the forfeiture. A blank reason line on a cancelled session is a **P2**.
- A **cash** visit refunded has no gateway behind it, so it becomes a hand-back for somebody to do: it shows on the admin's Cash Ledger until an admin confirms the cash was returned.

Then look at how the same refund reads to each party:

Screen	Expect
Admin → All Sessions	A chip naming the refund state. An unrefunded session shows nothing - not an empty chip.
Admin → the session drawer	The same state, plus when and by whom
The patient's Sessions card	A line answering <i>"am I getting my money back, and when"</i> , with the clinic's stated reason
The patient's Payments screen	The same refund on the matching row

One state that must say nothing to the patient: a decision that **nothing is owed**. The cancelled card already explains the window; repeating it as a refund line announces a refund to somebody who is not getting one.

Step 8.9 - Switch home visits off with a recommendation outstanding

Do this

- 1. Have QA Therapist A recommend **HV2** (the four-visit home programme) to Patient B, and approve it as the admin.
- 2. **Before** Patient B pays, switch **Home Visit** off in Settings.
- 3. As Patient B, try to buy it.

Expect. Refused, with *"Home visits aren't available right now. Please talk to your therapist."* An admin who switches home visits off has stopped the service, and a recommendation written before that must not stay purchasable.

Switch home visits back **on**.

Step 8.10 - Checkpoint

	Should be
Patient A	One online-paid visit (₹2,649 charged), one cash visit delivered
Cash ledger	One collection, one correction, both recorded
Waitlist	One entry for 560025
Refunds	One processed, one forfeited, one cash hand-back outstanding

10. Part 9 - The partner referral, end to end

What this part does. Follows one referred patient from the hospital's form to a delivered session and the commission it earns. This is the one flow where the clinic has to reach somebody who has no account yet, which is why a phone number is required.

Time. About 50 minutes.

Step 9.1 - Refer a patient

Who you are. QA Sunrise Hospital (qa.hospital@example.test , the generated password from Step 3.10).

Do this

- 1. Open /hospital/dashboard/refer .
- 2. Fill it in:

Field	Value
Patient Full Name	QA Referred Patient C
Patient Phone Number	pick the country, then 9876543210
Session Type	Online
Address	8, 100 Feet Road, Indiranagar, Bengaluru
Preferred Language	English
Medical Issue	Right-sided weakness following a stroke six weeks ago
Treatment Needed	Gait and balance retraining, twice weekly

1. Submit.

Expect

- A teal confirmation: *"Referral submitted - our team will review and reach out."*
- The form resets and Session Type returns to **Online**. The phone field clears with the rest.
- It appears under **Your Referrals** as **Pending Review**, and in the admin's **People → Partners** with the badge raised.
- **The Pincode field is not required** for an online referral.

Then try it with the phone blank.

Expect. Refused with **Enter the patient's phone number so our team can reach them.** The number is required because the clinic **rings this patient before sending the registration link** - they have no account to message.

Step 9.2 - Refer a home visit, and find the pincode rule

Do this. Submit a second referral for the same patient, choosing **Home visit**, leaving **Pincode** blank. Then **56003**. Then **560038**.

Expect

Value	Expect
blank	Enter the patient's 6-digit pincode for a home visit referral.
56003	the same refusal
560038	accepted

And check the switch reaches here too. As the admin, switch **Home Visit** off, then reload the referral form.

Expect. The **Home visit** option is **not offered at all**. A partner must not be offered a delivery mode the platform has not turned on. Switch it back on.

Step 9.3 - Watch the admin drive the pipeline

Who you are. The Master Admin, with the hospital's screen open in the other browser so you can watch it change.

Do this. Open **People → Partners → Patient Referrals** and find `QA Referred Patient C`.

Expect, before you touch anything. The patient's **phone number** and **preferred language** are printed directly under their name. This is the one flow where the clinic must reach somebody who has no account, and the number being a click away is the point. If the number is missing but the rest of the card renders, the database may simply predate that column - check the referral list itself is intact before reporting it.

Now drive each transition, checking the hospital's screen after each:

Do this	Hospital's Your Referrals should read
Nothing yet	Pending Review
Assign a therapist and a slot	Therapist Assigned
Send the registration link	Invite Sent
<i>(after Step 9.5)</i> the patient registers	Registered

Expect. The hospital sees **status only** - never the patient's clinical record, never a session note, never a health profile.

When you assign the slot, look at the control you are given.

Expect. The same compact calendar and hour chips the patient's own booking screen uses, obeying the **same 12-hour lead time**. An admin must not be able to promise a referred patient a slot the platform's own rule would refuse - two answers to "when can this be booked", in the one flow where the person choosing is not the person who lives with it. A raw date-and-time box here is a **P1**.

Step 9.4 - Try to withdraw at the wrong moment

Do this. As the hospital, withdraw a referral that is still **Pending Review**. Then try to withdraw the one that is **Invite Sent**.

Expect

- The pending one withdraws.
- The invited one is refused with `An invite has already been sent for this referral, so it can't be withdrawn`.
- Declining from the admin side **requires a reason:** `A reason is required to decline.`

Step 9.5 - Register Patient C, carrying the attribution

Two ways in. Do both, on two different referrals.

Way one - typing the code. As a signed-out visitor, open `/book`, complete Step 1, and on Step 2 enter:

Field	Value
Full Name	<code>QA Referred Patient C</code>
Email Address	<code>qa.patient.c@example.test</code>
Phone	<code>+91 98765 43212</code>
Password	<code>QaTest!2024pass</code>

Field	Value
Referral Code	Hospital A's code from Step 3.10
What would you like help with?	QA Neuro Rehabilitation

Tab out of the Referral Code field and read the line.

Expect. `Checking code...`, then in teal `Valid - referred by QA Sunrise Hospital`. Complete the booking and pay (`4111 1111 1111 1111` , Success).

And the two negatives:

Code	Expect
<code>ZZZZZZ</code>	Continue is blocked with <code>That referral code isn't recognized...</code>
blank	Nothing is blocked; the booking proceeds unattributed

Way two - the registration link. Copy the link from the admin's Partners screen and open it in a **private window**, then register.

Expect. The card is already associated with that referral, the account is created with a session immediately (no email step), and attribution is set **without the patient typing a code**. The referral becomes **Registered**.

The words matter here. This is a **registration link**, never an "invite link". An invite in this product is one patient telling another, which is a different feature with different money attached - and one back office cannot have two things called an invite.

Step 9.6 - Deliver a session and check the commission

Do this

1. As the admin, assign Patient C's paid session to **QA Therapist A** and have it delivered and completed (as in Part 5).
2. Cancel and **refund** a second paid session of Patient C's.
3. As the hospital, open **Earnings**.

Expect

- The completed ₹2,499 session contributes **10%** - `₹249.90`.
- The refunded session contributes **₹0**. A refund **reverses the partner's commission**, because the commission is a cut of money the clinic kept.
- The figures here match the admin's **Money → Breakdown** for the same range **exactly**. Two screens disagreeing about one commission is a **P0**.
- The sidebar's money word is **Earnings**, matching the therapist's. Not "Revenue & Payouts".
- Balances are **not** date-filtered; flows are - and each label says which it is.

Step 9.7 - Check the two states that must not collapse

This is subtle and it is worth doing carefully, because collapsing them hides real money.

Do this

1. Have **QA Therapist B** deliver a completed paid session for Patient C. Therapist B has a revenue share (55%), so first **clear it** so they have none configured.
2. Open the admin's **Money → Summary** and **Money → Breakdown**.

Expect

Case	What must happen
A patient not referred by any hospital	Hospital cut of 0 , and the appointment stays in the split
A patient referred by a hospital whose share is unconfigured	The appointment is excluded from the split entirely - it still counts in Gross, Refunds and Net, and contributes to none of therapist cut, hospital cut or clinic share
A session whose therapist share is unset	The same exclusion

- Every excluded appointment is surfaced as a **named count** and a named revenue figure, not silently dropped.
- **No percentage is guessed to make the numbers tie.** If a 0% hospital cut and an unconfigured hospital produce identical figures, the split has lost the distinction - **P0**.

Put Therapist B's share back to **55**.

Step 9.8 - Check the hospital is fenced in

Do this, each as **`QA Sunrise Hospital`**:

Try	Expect
Open Your Referrals	Only Hospital A's rows. Never Hospital B's.
Open <code>/admin/dashboard</code>	Redirected to `/get-started` - never to <code>/admin/login</code> . Naming the back office's door to somebody outside it is the failure.
Open <code>/patient/dashboard/health-profile</code>	Redirected away. No clinical data.
Open <code>/book</code>	The wrong-account panel, pointing at referring.

Then two console calls, signed in as the hospital:

```
// 1. Someone else's referral
const a = await fetch("/api/hospital/withdraw-referral", {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify({ referralId: "<a referral id that is not Hospital A's>" }),
});
console.log("withdraw:", a.status, (await a.text()).slice(0, 120));

// 2. A patient's uploaded report
const b = await fetch("/api/medical-documents/view", {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify({ documentId: "<Patient A's report id>" }),
});
console.log("document:", b.status, (await b.text()).slice(0, 120));
```

Expect. Both refused - **403** or **404**, and Hospital B's referral is unchanged. Neither answer names a table or a column. A **200** on either is a **P0** that stops the run.

Step 9.9 - Suspend the hospital

Do this. As the admin, suspend `QA Sunrise Hospital`. Then, in the hospital's still-open browser, reload the dashboard and run one console call.

Expect

- The dashboard redirects to `/account-suspended``.
- The API call is refused.
- **The suspension reaches further than the screen.** A suspended account whose cookie still works against the data layer is the gap this check exists for.

Restore the hospital.

Step 9.10 - Checkpoint

	Should be
Patient C	Registered through a referral, attributed to Hospital A, one completed paid session, one refunded
Hospital	Earnings showing ₹249.90, matching the admin's Breakdown
Referrals	One withdrawn, one declined with a reason, two registered
Exclusions	The unconfigured-share case counted and named, never estimated

11. Part 10 - The books

What this part does. Reads everything the run has earned and paid out, and checks the figures agree with each other and with the rows behind them. By now there is real money in the database: two consultations, a programme, two home visits, a referral, a refund and a cash collection.

Who you are. The Master Admin.

Time. About 50 minutes.

Step 10.1 - Read the Money screens in order

Do this. Open each Money screen and read the whole of it before moving on: **Summary, Breakdown, Costs, Payouts, Business Health.**

Expect, on every one of them

- An **alerts strip** at the top: payout requests waiting, cash a therapist is holding, refunds to hand back by hand, payments attached to nothing. A zero row is **dropped**, not shown as a zero.
- A **glossary** at the foot, and an **(i)** on each figure giving the same definition beside the number that needs it.
- Each figure carries a **scope chip** saying whether it moves with the dates in view (**range**), is true this instant (**now**), or is a rate (**setting**). Narrowing the date range and watching one figure fall while the one beside it holds still is correct **only** if the chips say so.
- A figure appears **once per screen**. Summary printing Net revenue twice is the defect this rule exists for.

Step 10.2 - Check the two identities hold

These must be true on every range you pick. Check them on at least two: all time, and this month.

$$\begin{aligned} \text{net revenue} &= \text{gross revenue} - \text{refunds} \\ \text{clinic share} &= \text{splittable net} - \text{therapists' share} - \text{partners' share} \end{aligned}$$

Do this. Read the figures off **Summary** and do the arithmetic by hand.

Expect. Both hold exactly. Not approximately - **exactly**. If the clinic share is labelled "approximate", or if the numbers only tie after rounding, something is being guessed, and guessing a percentage to make the numbers tie is the thing this part exists to catch.

Then check the three split rules, each on a real row in your data:

Rule	Check it with
A therapist's share is earned by delivering , not by being booked	The forfeited late cancellation from Step 8.8 - it is paid and not completed, so it adds nothing to any therapist's share
A home visit's travel fee is part of the therapist's share , never revenue	The ₹150 from Step 8.1
Refunds reverse the partner's commission, not the therapist's	Patient C's refunded session from Step 9.6 - the therapist never earned a share on it, and the hospital's cut reverses

Step 10.3 - Open a total

Do this. On **Summary**, tap **See the sessions** on each of the four split figures.

Expect

- A list of **exactly** the rows behind that figure.

- The lines **sum to the total** that opened them. A drill-down that disagrees with its own total is worse than none - it makes a correct figure look wrong.
 - It exports like every other table, in both CSV and PDF, and the two describe the same rows.
 - Net revenue carries a comparison against **the same number of days immediately before** - not last calendar month against a 30-day window, which would move the figure by the number of days rather than by the business.
 - With a **zero baseline** there is **no percentage** at all. `+100%` and `∞` are both lies.
 - A move under half a percent reads **level** rather than drawing an arrow over noise.
-

Step 10.4 - Pay a therapist

Do this

1. As **QA Therapist A**, open **Earnings** and request a payout.
2. As the admin, open **Money → Payouts**, read what Therapist A is owed, and settle it.

Expect

- What is owed counts **completed, paid** sessions only, at 60% for online and **65%** for their home visits, plus travel in full.
- **The cash they are holding is netted off the transfer.** Therapist A collected cash at Step 8.5: the amount actually transferred is reduced by it, and **those visits are marked remitted in the same run**. If they are not, the same rupees are deducted again on the next payout and the Cash Ledger goes on asking somebody to chase money already recovered - **P1**.
- "Owed to therapists" is **all-time and not date-filtered**. Scoping a balance to the range in view lets an admin read "nothing owed" off a quiet week while a real debt sits outside the window.
- The settlement is recorded in **Logs → All Activity**, and the log row was written **after** the claim - so a settlement that lost a race can never appear in the log.

Then the edge case. Have a therapist hold **more cash than they are owed** (record a large cash collection against Therapist B).

Expect. The transfer **floors at zero**, the difference stays as an amount still owed **to the clinic**, and those collections stay open on the Cash Ledger for a person to chase. A negative transfer is a **P0**.

Step 10.5 - Work the refund queues

Do this. Open **Money** and **Today**, and read the counts.

Expect

- **Refunds to hand back by hand** counts **cash visits and sessions alike**. A session refunded by hand used to be work no screen could see.
- **Failed refunds** are counted **separately** - the gateway said no, and nothing in the clinic's screens will move that without the patient. The two are separate because the work is: one is "go and hand over cash", the other is "find out why the gateway refused".
- Tapping either count opens **exactly** the rows it counted.
- A **failed** refund is also a pinned item on the patient's own dashboard, because they are the one out of pocket.

And check the count is not double-counting. The Money strip and the Cash Ledger beneath it must agree on how much cash is outstanding. A strip reading twice the ledger's figure means cash home visits are being counted in two places - **P1**.

Then check the export follows the scope. Sign in as **QA Admin Operations**, open All Sessions, and export it.

Expect. The amount and the refund columns are **absent from the file**, exactly as they are absent from the screen. A scope enforced in the markup and not in the file the markup produces is not enforced.

And one filter worth checking by name. On All Sessions, set the payment filter to **Refunded**.

Expect. Rows come back. If it silently returns an empty table, the filter is matching a value the column can never hold - a refund lives on its own field, not on the payment status.

Step 10.6 - Record costs and read the profit

Do this. Open **Money → Costs** and add three expenses:

Description	Kind	Amount (₹)	Incurred on
QA Clinic rent September	pick the rent/premises category, fixed	25000	the 1st of this month
QA Software subscriptions	software/tools, fixed	4000	the 3rd of this month
QA Marketing test spend	marketing, fixed	6000	last month, deliberately outside a this-month range

Expect

- Each cost carries a **kind**, deciding whether it sits above or below the gross-profit line and whether it is added back inside EBITDA. Nothing is inferred from the wording.
- Set the range to this month: the third row is **excluded**, because it is dated by when it was incurred rather than when somebody typed it in.
- **Operating profit** appears - clinic share less the gateway fee and these costs. It is the **only** figure allowed to be called profit, and nothing here is post-tax: a figure labelled "net profit" is a **P2**.
- With **no** costs recorded for a range, Operating profit is stated as a **ceiling** and the screen says so, rather than implying a number it cannot know.

Also read the discount line. *Discounts given* is **reported, never deducted**. A discount means less was collected, so it is already inside gross revenue as a smaller number; subtracting it from profit would count it twice. If Operating profit falls when you give a discount at Part 11, that is a **P0**.

Step 10.7 - Read Business Health

Do this. Open **Money → Business Health**.

Expect

- Seven figures: return on investment, return on ad spend, working capital, gross and net margin, EBITDA, break-even and revenue run rate.

- **A figure that cannot be worked out is a sentence, not a zero.** With nothing invested, ROI says which input is missing and links to the screen that takes it. A zero here would be read as a measurement and acted on.
- Each carries its **formula** and **where each input came from** behind its (i).
- Revenue and the split match **Summary** exactly - both read the same source.

Then give it the three things it cannot derive. Open **Money → Your Numbers** and enter:

What	Value
Invested	500000 , life 36 months
Ad spend	one campaign, 6000 , this month
What the clinic owns and owes	a dated snapshot, today

Expect

- ROI and EBITDA now compute.
- **Advertising revenue is traced or it does not exist.** A campaign is worth the net revenue of bookings that claimed **its promo code**. Spend nobody can trace is stated **separately** and held out of the division - leaving it in the denominator reports a campaign as a failure purely because nobody tagged it.
- **Working capital counts money taken for sessions not yet delivered** as a liability, valued at what was **actually paid**, never at the live catalogue price. Patient A's unbooked session from Step 7.9 is in there.
- A snapshot is dated: entering this month's figures does not erase last month's.

Then narrow by a dimension - filter to one therapist.

Expect. An amber line saying the costs are **not** narrowed with the revenue. Rent is not attributable to a therapist, so a filtered profit figure compares one slice's revenue with the whole clinic's costs, and the screen says so rather than leaving somebody to work it out by accident.

Step 10.8 - Flip the ledger switch

Do this

1. Note Patient A's programme balance on all four surfaces from Step 7.9.
2. Open **Settings → Programmes & Home Visits** and switch **Session Balances From The Ledger on**.
3. Read the same four surfaces again.

Expect. Every one of them reads the **same balance as before**. The switch changes which record the app believes, and both are written either way - so flipping it must move nothing. A surface that disagrees after the flip is a **P0**, and it is exactly the disagreement the switch exists to make visible.

Then check the reconciliation report. Open **Settings → System Health → Books & Sessions Agree**.

Expect. It **reports** and never repairs. A silent auto-fix on a money record is how a discrepancy becomes permanent. If it lists a disagreement, that disagreement is the finding - write it up rather than clearing it.

Leave the switch **off** when you are done, unless you were asked to leave it on.

Step 10.9 - Adjust a balance, and try to adjust history

Do this

1. Open **Catalog → Purchases** and find Patient A's programme.
2. Grant 2 extra sessions with the reason `Goodwill` (seven characters).
3. Then with the reason `Goodwill after a cancelled session.`
4. Then try to **reverse** one, and to **revive** an expired entitlement.

Expect

- The seven-character reason is **refused** - a free-form adjustment is the only kind requiring a reason, and the floor is **ten characters**, enforced deep enough that no caller can get round it.
 - The valid one succeeds and **appends** a row. The balance moves.
 - An admin can change **any balance** and **cannot change any history**. The ledger only ever grows.
 - Every adjustment is in **Logs → All Activity**.
-

Step 10.10 - Read System Health

Do this. Open **Settings → System Health** and read all five checks.

Expect

- Each has a **word** as well as a colour: `Healthy`, `Needs a look`, `Needs you now`, `Not set up`, `Not checked`.
- **`Not set up` and `Not checked` are not faults.** An owner who never wired Google up has not got a problem, and painting that red is how red stops meaning anything.
- **Anything not healthy carries numbered steps** the owner could follow alone. A red card with no way out is a **P1**.
- The teaching text is behind the **(i)**, not on the card.
- The sidebar badge **counts checks, not rows** - so a failure with no rows behind it (a missing webhook secret, a dead Google credential) still badges 1 rather than 0.
- A **red** check puts one line on the admin's Today screen. An amber one does not - a banner that is usually there is a banner nobody reads.
- The Google card prints **when it was last checked**, and that relative time is rendered after the page loads rather than on the server.

One check worth reading closely. If `RAZORPAY_WEBHOOK_SECRET` is unset, System Health says so - and it matters: without it, a patient who pays and closes the tab leaves a paid order against an unpaid booking, and nothing else in the app will notice.

Step 10.11 - Payment integrity

Money can arrive twice, arrive late, or not really arrive at all. Three checks, all from the Razorpay dashboard's own webhook log.

An authorization is not a payment. Razorpay can *authorize* a card - a hold - and capture it seconds later. A hold that is never captured is voided and refunded a few days on, so nothing may be fulfilled against one.

Do this. From the Razorpay test dashboard, send a ``payment.authorized`` event for a booking that is still unpaid, then look at the booking.

Expect

- The booking is **still unpaid**. Nothing is confirmed, no calendar event is made, no invite reward is settled.
- The event **is** recorded in the webhook trail even so - the record is worth more than the one saved delivery.
- Then send ``payment.captured`` for the same order: *now* everything happens. If the authorization alone marked it paid, that is a **P0** - the app would be delivering care against money that can still evaporate, and nothing walks it back.

A duplicate webhook changes nothing twice.

Do this. Re-send the same `payment.captured` event two or three times.

Expect. One payment recorded, one booking confirmed, **one** calendar event. The second delivery finds the order already captured and does nothing. Two `payments` rows for one order, or two calendar invites reaching the patient, is a **P0**.

And the callback and the webhook must agree on the figure. Pay for a **home visit** in the browser and let the callback confirm it, then read the recorded amount.

Expect. It is the **whole** of what the gateway took - the visit **plus travel**, `₹2,649` at Step 8.1's prices - not the service line alone. One booking recorded two different ways depending on which arrived first is the disagreement this record exists to settle.

Finally, check the unmatched-payment alert. Money's alerts strip counts payments attached to nothing. It should read **zero** unless you have deliberately made one.

Step 10.12 - Checkpoint

	Should be
Identities	Both hold exactly, on two ranges
Payout	One settled, cash netted off, those visits marked remitted
Costs	Three recorded, one correctly outside a this-month range
Business Health	Seven figures, with the three typed inputs in place
Ledger switch	Flipped and flipped back, no balance moved
Adjustments	One refused for a short reason, one applied

12. Part 11 - The four ways money comes off

What this part does. Switches on each of the four acquisition discounts in turn and proves the rule they all share: **the browser sends a name, never a figure**. Every amount comes from a row an admin created.

Time. About 60 minutes.

You need patients who have never paid. Three of these apply only to somebody's first paid session, and Patients A and C have both paid. Register two fresh ones now, through `/patient/register`, and approve them as the admin: | | **Patient D** | **Patient E** | | --- | --- | --- | | Full Name | QA Patient D | QA Patient E | | Email | qa.patient.d@example.test | qa.patient.e@example.test | | Phone | +91 98765 43213 | +91 98765 43214 | | Password | QaTest!2024pass | QaTest!2024pass |

Step 11.1 - The first-session offer

Do this

1. Open **Settings → Offers & Discounts**. Switch the **first session offer** on and set it to take **₹500** off.
2. As **Patient D**, go to `/book` and reach Step 3 against **QA Back & Spine Care** (₹1,999).
3. Read the Session Fee and the button.
4. Pay.
5. Book a **second** session as Patient D and read Step 3 again.

Expect

- Step 3 quotes **₹1,499**, and the button says the same. **The figure on the button and the figure Razorpay opens with must match** - this is the exact bug that motivated the whole quote module: the wizard printed ₹1,999 while checkout quietly applied the offer behind it.
- Step 5 quotes the **full ₹1,999**. A patient is new exactly once, and that is asked of the database rather than remembered in a browser.
- The offer is **video consultations only**. Check a home visit still quotes travel and no offer.

Then check a stranger is quoted it too. In a **private window**, signed into nothing, open `/book` and reach Step 3 for the same condition.

Expect: ₹1,499. A visitor with no account yet is exactly who a first-session offer is for - the account, the booking and the payment all happen further down that same screen. Quoting them list price and then charging the offer is the same bug pointing the other way.

Step 11.2 - The goodwill adjustment

Do this

1. As **Patient E**, create a booking against **QA Knee & Joint Care** (₹1,799) but **do not pay**.
2. As the admin, open that session and apply a goodwill discount of **₹300** with the reason **Rescheduled twice by the clinic**.
3. Try it with the reason **Sorry** (five characters).
4. Try it with an amount of **₹1,799** - the whole price.
5. As Patient E, pay.

Expect

- Step 2 applies, and writes a row in **Logs → All Activity**.
- Step 3 is **refused** - ten characters minimum, enforced both by the route and deeper down, so no caller gets round it.
- Step 4 is **refused**. This is a number a person typed with the price on screen beside it, so a figure at or above the price is a typo - and quietly charging ₹1 because of it is far worse than saying no. Goodwill is

the one rule still floored above zero.

- Step 5 charges **₹1,499**.

Then try it on something already paid for.

Expect. Refused. A discount on something already paid for is a refund, and refunds have their own route, their own gateway call and their own audit trail.

Step 11.3 - The promo code

Do this

1. Open **Money → Costs** - the campaigns sit beside the figure they cost, not in Settings.
2. Switch **promo codes** on.
3. Create a campaign:

Field	Value
Code	QASPINE20
Kind	percentage
Amount	20
Window	today to a month from today
Redemption cap	2

1. As a patient with no prior paid session, reach Step 3 on a ₹1,999 condition, type , and read the quote.
2. Pay.
3. Have a second patient claim it and pay.
4. Have a **third** patient try to claim it.

Expect

- Step 4 quotes **₹1,599**. The request carries the **code**, never an amount - the kind, the figure, the window and the caps all come from the row.
- Step 7 is **refused**: the cap means two, and it holds even while claims are open, because it is enforced under a lock rather than by a count taken a moment earlier.
- A claim that is never paid for **stops counting** after a checkout hold - abandon a checkout with the code applied, wait out the hold, and the slot is free again. Nothing writes an "expired" status anywhere.

Then the three refusals:

Try	Expect
A code that does not exist	Refused with a readable message
A paused campaign	Nothing applied, and the screen says which
A campaign whose window has passed	The same, naming the reason

And the one that matters most. Apply a valid code at Step 3, then - before paying - have the admin **pause** the campaign. Now pay.

Expect. Checkout **refuses** rather than quietly charging list price. The patient was shown a figure with the code applied, and **taking more money than somebody was quoted is the one outcome a payment screen must never produce.**

Then try to delete a code that has been used.

Expect. Refused - it can be **paused**, never deleted. A paid session pointing at a campaign nobody can name cannot answer which rule gave the money away.

Step 11.4 - The patient invite

Do this

1. Open **Settings → Offers & Discounts**, switch **patient invites** on, and set: the friend gets **₹200**, the inviter gets **₹200**, ceiling **10** per patient.
2. As **Patient A** (who has paid), find their invite code.
3. As a brand-new patient - register `QA Patient F`, `qa.patient.f@example.test`, `+91 98765 43215` - claim Patient A's code at checkout and pay.
4. Look at Patient A's next booking.

Expect

- Patient F's first session is **₹200** cheaper.
- **Patient A's half is earned when Patient F's session is paid for, not when they signed up.**
Check Patient A's reward appears only after the payment lands. A reward that pays out on signups is a reward for creating accounts, and somebody will.
- Patient A's next booking is **₹200** cheaper, once.

Then the four refusals:

Try	Expect
Patient A claims their own code	Refused
Patient F claims a second invite	Refused - a patient is new exactly once
Patient C , who has already paid, claims one	Refused
Somebody claims a code from an inviter already at the ceiling	Refused, and worded so it does not tell them about somebody else's account

And check a promise already made is kept. With an unspent reward outstanding, **lower** the reward to ₹100, then let the patient spend the old one.

Expect. They get the **₹200** that was promised. Amounts are snapshotted when claimed - lowering the reward later must not lower what was already agreed. Switch invites **off** entirely and check the same: the switch stops new claims, it does not withdraw a promise.

Step 11.5 - Check they never stack

Do this. Set up a patient who qualifies for **two at once** - a first-session offer of ₹500 and a promo code worth ₹300 - and read Step 3.

Expect

- **One** discount applies: the **largest**, so ₹500. They never stack.
- The patient pays the **lowest** price any of the rules would have given. The clinic agreed to every one of those prices, so charging a higher one because an admin tried to help would be perverse.
- Where two are equal, the **more deliberate** decision wins: goodwill first, then the code the patient typed, then the campaign that runs itself.
- The losing code is **released** - check it has not counted against its own cap for nothing.

And check travel is never discounted. Apply any of these to a **home visit** and read the breakdown.

Expect. The discount comes off the **service line only**; travel is added back afterwards at full price. Discounting travel makes the therapist fund their own transport to subsidise the clinic's marketing - **P0**.

Step 11.6 - The free booking

Do this

1. Create a promo code `QAFREE100`, **100%** off, cap `1`.
2. As a patient with no prior paid session, apply it at Step 3.

Expect

- The total reads **Free**, the lock line changes to *"Nothing to pay - your discount covers this session in full"*, and the button reads **Confirm booking - free**.
- Tapping it books the session with **no Razorpay screen at all**.
- **Being charged ₹1 instead is a P0.** That was the old behaviour, and it charges a figure nobody was quoted.
- The session appears on the patient's dashboard exactly like a paid one - confirmed or pending as usual.

Then prove the server decides it is free, not the browser. With **no** discount running, signed in as that patient:

```
const r = await fetch("/api/appointments/confirm-free", {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify({ appointmentId: "<an unpaid appointment id of yours>" }),
});
({ status: r.status, body: await r.text() });
```

Expect. 409, *"This booking still has an amount to pay."*, and the booking stays unpaid. If a booking can be confirmed free by asking, every session in the app is free - **P0**.

Then tap the free confirmation twice on a genuinely free booking.

Expect. Success both times, **one** booking. It is idempotent.

And check what a free booking records. As the admin, open that session:

Check	Expect
A <code>payments</code> row	None. That table is the record of money that moved, keyed on the gateway's own ids; a collection of zero has neither.
The amount recorded	Zero , with all four discount facts beside it - list price, amount off, which rule, and why
Everything else	Still happened: assignment, the calendar event, any invite half settling, the patient's approval

Step 11.7 - Check the books can tell cheap from discounted

Do this. Open **Money** → **Costs** and read *Discounts given*.

Expect

- A figure **split by rule** - how much the first-session offer cost, how much the promo codes cost, how much goodwill cost, how much invites cost.
 - It is **stated, never subtracted** from profit. Compare Operating profit before and after giving a discount: it must fall by the revenue not collected, and **not twice**.
 - Every discounted session records **four facts** - the list price, the amount off, which rule, and why. A discount implemented by simply charging less leaves the books unable to tell "*we sold this cheap*" from "*we discounted it*", and that difference is the one number that decides whether an offer continues.
-

Step 11.8 - Put the switches back

Unless you were asked to leave them on, return each to how Step 2.7 found it:

Setting	Back to
First session offer	off
Promo codes	off
Patient invites	off
Any campaign you created	paused, not deleted, if it has been claimed

Step 11.9 - A goodwill discount collected at the door

This is the one combination where a discount meets cash, and it has been wrong before: the discount was given, and the full price was then recorded as the cash taken.

Do this

1. As a patient, book a **home visit** choosing **Pay at the visit** (as at Step 8.4).
2. As the admin, apply a goodwill discount of **₹300** to that visit with the reason `Long wait for the first appointment.`
3. As the admin, mark it **paid by cash**.
4. Read three things: the amount recorded against the visit, the therapist's **Cash Ledger** figure, and **Money** → **Summary**'s gross revenue.

Expect

- All three reflect the **discounted** figure. The discount facts on the row - what the list price was, how much came off, which rule, and why - must describe a reduction the recorded amount **actually shows**.
 - Recording the full price here overstates the cash ledger *and* gross revenue by exactly the amount given away, and asks the therapist to hand over money nobody collected. **P0**.
-

Step 11.10 - Checkpoint

	Should be
Each of the four discounts	Applied once, refused where it should be
Stacking	Proved not to
Travel	Never discounted
Free booking	One, with no payment row and all four facts recorded
Money → Costs	Discounts given, split by rule, not deducted from profit

13. Part 12 - Who may do what

What this part does. Checks the access model where it is actually enforced - in the routes, not in the sidebar - plus the settings that change other screens, the log, and signing in as somebody else.

Time. About 60 minutes.

Every check here needs a second browser. Each scoped admin is a different session. Sign in as one, run its checks, sign out, and move on.

Step 12.1 - Each desk lands on its own dashboard

Do this. Sign in as each of the four admins in turn and read the Today screen **before** touching anything.

Signed in as	Expect the dashboard to lead with
Master Admin	Everything, unchanged
qa.admin.ops@example.test	Unassigned sessions
qa.admin.finance@example.test	What is owed to therapists
qa.admin.clinical@example.test	Recommendations a patient is waiting on

Expect, on each

- The dashboard **names itself twice**: in the sidebar brand above *Admin Panel*, and again as the eyebrow over the section heading - Master Admin, Operations, Finance, Clinical. Twice because the sidebar collapses on a phone and the header is on every screen at every width. Four dashboards that all said "Admin Panel" and differed only in which entries were missing is the failure.
- Each of the three limited desks gets a **"Your access"** card naming which sections that desk covers. A Master Admin does **not** - a shorter sidebar with no explanation reads as a fault, and a full one needs no note.
- Every quick action lands somewhere that desk can actually open.** Tap all of them. An action that lands back on Today is the exact defect this rule exists for: a finance admin's "All sessions" used to do that silently.
- The "needs you" figure **agrees with the queue list beneath it**, and counts only queues that desk can **work** - a section they can only read holds no work for them.

Step 12.2 - The sidebar is presentation; the routes are the rule

This is the most important check in the part. A hidden button proves nothing.

Do this. Signed in as `qa.admin.finance@example.test`, open DevTools → Console and run the sweep:

```
const routes = [
  "/api/admin/approve-account",
  "/api/admin/assign-therapist",
  "/api/admin/create-booking",
  "/api/admin/cancel-appointment",
  "/api/admin/update-treatment-category",
  "/api/admin/review-care-plan",
  "/api/admin/save-therapist-availability",
  "/api/admin/set-admin-scope",
  "/api/admin/clear-activity-log",
  "/api/admin/activity-log",
];
console.table(await Promise.all(routes.map(async (route) => {
  const r = await fetch(route, {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: "{}",
  });
  return { route, status: r.status, body: (await r.text()).slice(0, 100) };
})));
```

Expect: 403 on every row. Then repeat the whole sweep signed in as **Operations** and as **Clinical**.

A `400` anywhere in the table is its own finding. It means that route read the request body **before** deciding whether the caller was allowed - the check ran after the parsing. Report it against this step.

Then check the one `view` grant is real. As **Finance**:

Try	Expect
Open Sessions → All Sessions	It opens . Finance reads Sessions - the question "what was this ₹1,200 for?" was answerable only by asking somebody else.
Cancel or assign a session from that screen	The control does not render . A control an admin's scope cannot call must not be on screen, or they get a 403 with nothing to explain it.
Call <code>cancel-appointment</code> from the console anyway	403.

That combination - open the screen, cannot change a row - is what makes `view` a real level rather than a label.

Step 12.3 - Complete a session from the wrong desk

Do this. As **Finance**, find a completed-eligible session and try to mark it complete, both from the screen and from the console:

```
const r = await fetch("/api/appointments/complete-session", {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify({ appointmentId: "<a paid, past session id>" }),
});
({ status: r.status, body: await r.text() });
```

Expect. Refused. Completing a session is what creates the payout obligation, and Finance holds Sessions at `view` precisely so the person reconciling the books cannot change what they are reconciling. The buttons are hidden on the same test.

Step 12.4 - Change a setting and watch the feature change

Settings are only real if something downstream moves. Check each of these, then put it back.

Change	Where	Then check
Online booking lead time <code>12</code> → <code>48</code>	Settings → Booking Rules	<code>/book</code> now offers nothing sooner than 48 hours out, and a direct booking at the old boundary is refused 409 with <code>Please pick a slot at least 48 hours from now.</code> Existing bookings are untouched.
Online cancellation window <code>24</code> → <code>72</code>	Settings → Booking Rules	Step 3's notice re-words itself, and a session 48 hours away now falls inside the window: the dialog says it will not be refunded, and no refund is processed. The home-visit dialog must not change - it reads its own setting.
Booking languages: add <code>Hindi</code> and <code>Kannada</code>	Settings → Booking Rules	Three chips on Step 1, in order, first auto-selected. Then remove them all: refused with <code>Keep at least one language - booking needs something to offer.</code>
Automatic therapist assignment on	Settings → Programmes & Home Visits	A newly paid session assigns itself only when exactly one therapist is unambiguously free, or when it is the one the patient asked for. With two free, it stays in the queue - assigning the wrong clinician is far worse than the wait.
Session Completed cutoff	Settings → Booking Rules	Every join control past it reads Session Completed , on all four dashboards including the admin's
Idle timeout <code>1</code> minute	Settings → Booking Rules	A patient left idle is signed out to their own login page. <code>0</code> means off. Admins are exempt entirely - check an idle admin is not signed out.

**And one regression that must not happen. Change QA Therapist A's roster* - remove a whole morning - then open `/book` as a patient.*

Expect. `/book` offers **exactly the same times as before.** The roster is the clinic's planning record for who can be *offered* work; it deliberately does not filter the patient's picker, which applies the lead-time rule alone. A picker that narrows when a roster changes is a **P1** here, not an improvement.

Then check the roster did not touch a booking. Remove hours that an existing session sits in.

Expect. The screen **names who is affected** and says the session stays as booked. Nothing cancels, moves or flags it. The booking wins.

Step 12.5 - Sign in as somebody else

Do this. As the **Master Admin**, open Patient A's profile and use the control that opens their dashboard as them.

Expect

- You are asked for a **reason**, at least ten characters. Try `test` - refused. Use `Patient reports the booking screen fails on submit.`
- The browser genuinely **becomes** that account - same routes, same data, same controls, every write real.
- An **amber bar sits above every dashboard screen**, naming the account, saying the actions are real, counting the window down, and carrying **Exit**.
- Exit** puts you back as yourself.

Then the four refusals:

Try	Expect
As Operations, Finance or Clinical	Refused. This is full scope only - a section scope would hand it to whoever can edit a phone number.
Opening another admin as somebody	Refused. That is one admin using another's authority.
Opening a suspended account	Refused.
Leaving the tab open past the window	The session ends - a forgotten tab is an open window into a health record.

And check the record. As the Master Admin, confirm a row exists naming who, whom, when and why - **written before the swap**, so a session with no record behind it cannot exist. Everything done during the window is written **as that user**: no column anywhere says an admin was at the keyboard, which is a real cost of the swap and is why the window's own start and end times are the only thing a later reader can line an action up against.

Step 12.6 - The log

Who you are. The Master Admin - **Logs** is this desk's alone.

Do this

- Open **Logs → All Activity**.
- Search for a patient's name. Filter by type. Set a date range.
- Tap any row.
- Tap the **subject** inside that row's detail.
- Export it, both formats.

Expect

- Every action this run has taken is there: approvals, assignments, the settlement, the goodwill discount, the recommendation decisions, the credit adjustment, the impersonation, the reset from Step 1.2.
- A row's detail says **what changed from what**, not merely that something changed.
`patient.update_contact` names the old and the new address - an entry saying a sign-in address was altered without saying what it had been is unusable for the one question it gets asked.

- **Nothing is dropped:** an unrecognised field is listed plainly, with the raw data behind a toggle.
- Values are **read, not printed raw** - money as rupees, timestamps as dates in clinic time, booleans as Yes/No, an absent value as a dash. The word "paise" never reaches the screen.
- The **four note routes record a length, not the text** - a note about one patient must not be reproduced across the back office.
- **No generated password appears anywhere in the log.**
- Step 4 opens that subject's whole timeline, keyed on the **id** rather than the name - so a patient renamed between two entries is still one person. An entry with no subject id offers no timeline rather than one built by guessing.
- Opening the timeline **closes** the entry behind it. Two stacked dialogs over a table leave a reader unable to tell which Escape closes what.

Then try to clear it.

Expect

- The **Clear** button is locked until a copy has actually been **downloaded** - not a checkbox saying one was. A record that is gone and was never kept is destroyed; one downloaded first has only been moved.
- A cutoff inside the **last 30 days** is **refused**, at every setting. The newest month - where anything worth hiding would be - is out of reach.
- The clearing **logs itself**, with the cutoff and the count, and that row is inside the protected window, so a clear can never remove the record of a clear.
- The count of what is about to go is taken **server-side**, not from the page you are looking at.

And check the three other desks cannot open it at all. As Operations, Finance and Clinical:

Try	Expect
The Logs entry in the sidebar	Absent
<code>?section=logs</code> in the URL	Lands somewhere else, and says why in one amber line rather than silently redirecting
The two Logs routes from the console	403

But each of them does read their own desk's work. Open **Today → Activity** as Operations.

Expect. Entries whose action belongs to a section they can work **and** whose actor sits at that desk. It will look **sparse** - in a small clinic the Master Admin does most of the work - and the screen says the list is their desk's rather than showing an empty screen that reads as "nothing happened".

Step 12.7 - Suspend and delete

Do this

1. As the Master Admin, **suspend** `QA Admin Operations` from Settings → User Access.
2. In their still-open browser, reload and run one console call.
3. Try to suspend **yourself**.
4. Try to narrow the **last** Master Admin.
5. Try to **delete** an account with history - Patient A.
6. Create a throwaway account and delete **that**.

Expect

- Step 2: locked out on the screen **and** at the route. A suspension that only the app enforces leaves a valid cookie reaching the data layer directly - this check is why both halves exist.
- Steps 3 and 4: both **refused**. Nobody changes their own scope, and the last Master Admin who can still sign in cannot be narrowed - a single mis-click would lock everyone out permanently.
- Step 5: **refused, with the counts named** - how many sessions, purchases, payments and records point at that account - and **suspension offered beside them**. It is not a policy: deleting "properly" would mean deleting the money and the clinical record with it.
- Step 6: deletes. The audit row is written **before** the delete, since afterwards there is no row left to name.
- Deleting is **full scope only**, checked directly - every desk that manages People can already suspend, and this one is irreversible.

Restore `QA Admin Operations`.

Step 12.8 - The doors a stranger can reach

Do this. In a **private window**, signed into nothing, run the anonymous sweep:

```
const routes = [
  "/api/appointments/create",
  "/api/appointments/cancel",
  "/api/patient/condition-profile/submit",
  "/api/therapist/save-availability",
  "/api/therapist/care-plan/submit",
  "/api/hospital/withdraw-referral",
  "/api/admin/approve-account",
  "/api/admin/settle-therapist-payout",
  "/api/medical-documents/view",
  "/api/razorpay/create-order",
];
console.table(await Promise.all(routes.map(async (route) => {
  const r = await fetch(route, {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: "{}",
  });
  return { route, status: r.status, body: (await r.text()).slice(0, 100) };
})));
```

The private window is the test. Running this where you are signed in sends that user's cookie and proves nothing.

Expect. 401 or 403 on every one. None returns 200, and none leaks data in the error body.

Then four malformed bodies at one route:

```
const route = "/api/appointments/create";
const bodies = {
  "invalid JSON": "{not json",
  "empty body": "",
  "an array": "[]",
  "a bare string": "\"hello\"",
};
console.table(await Promise.all(Object.entries(bodies).map(async ([name, body]) => {
  const r = await fetch(route, {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body,
  });
  return { name, status: r.status, body: (await r.text()).slice(0, 100) };
})));
```

Expect. Always **4xx**, never a **500**, and never a crash. A 500 here means the body was parsed without being caught - the request was the caller's to get right, so the honest answer is 400.

And check the tampering refusals:

Try	Expect
As a patient, send <code>{"role": "admin"}</code> to a profile-update route	Ignored or refused. Never trust a role from a browser.
As a patient, send somebody else's appointment id to <code>cancel</code>	Refused
Send a booking amount in the body of <code>create-order</code>	The amount is re-derived server-side from the category row, whatever you send

Step 12.9 - Knock on a public door repeatedly

Do this. In a private window, call the area lookup about twenty times in a row:

```
for (let i = 0; i < 20; i++) {
  const r = await fetch("/api/home-visit/check-area", {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({ pincode: "560038" }),
  });
  console.log(i, r.status);
}
```

Expect

- After a while, **429**.
- The message carries **no numbers** and **no blame** - a limit is reached by a shared office address or a connection retrying far more often than by anybody doing anything wrong, and "Too many attempts" reads as an accusation to all of them. The concrete wait, if any, is composed separately from a measured value.
- **A 429 is not a "no"**. Check the screen that calls this: a patient holding a good registration link must never be told it has **expired**, and a patient must never be told the clinic does **not visit their address**, because the app could not ask. The honest state is "we could not check", and it is a third answer rather than a falsy one. Either of those wrong messages is a **P1**.

Step 12.10 - Check the back office is never named to outsiders

Do this

As	Open	Expect
A signed-in patient	<code>/admin/dashboard</code>	Redirected to <code>`/get-started`</code> - never to <code>/admin/login</code> , which would confirm the back office exists and name its door
Signed out	<code>/admin/login</code>	It renders (it is the real door) but is marked not to be indexed
A signed-in therapist	<code>/patient/dashboard</code>	Bounced. Each role reaches only its own.

And one response body worth reading. In a private window with no session at all:

```
const r = await fetch("/api/admin/stop-impersonation", { method: "POST" });
({ status: r.status, body: await r.text() });
```

Expect. Whatever it answers, it must **not** name `/admin/dashboard` or `/admin/login` to a stranger. A route can leak the back office in what it says as well as in what it lets you do.

Step 12.11 - Check the response headers

Not everything that protects a patient is on screen. These ship on every response and cost nothing to check.

Do this. Open any page, DevTools → **Network**, click the document request, and read **Response Headers**.

Expect to find all of these:

Header	Should be
<code>X-Frame-Options</code>	<code>DENY</code>
<code>X-Content-Type-Options</code>	<code>nosniff</code>
<code>Referrer-Policy</code>	<code>strict-origin-when-cross-origin</code>
<code>Permissions-Policy</code>	present
<code>Strict-Transport-Security</code>	present (on an HTTPS deployment)

Why each matters here. Without the first, the admin dashboard and a patient's health profile can be framed by any site on the internet. Without the third, a referral token or an appointment id sitting in a URL travels to third parties as a full referrer.

One you should expect to see complaints about. A content-security policy may be present in **report-only** form, logging violations without blocking them. That is deliberate: Razorpay's checkout injects its own script and frame, and a policy written blind takes down checkout, which is the one failure a payment screen must not have. Console warnings from a report-only policy are **not** a defect - note them and move on.

Step 12.12 - A patient's route answers a patient

Three routes on the patient dashboard used to answer a therapist or a hospital with a **200**. Nothing cross-account leaked - each acts on the caller's own row - which is exactly why it survived unnoticed.

Do this. Sign in as **QA Therapist A**, then run:

```
const routes = [
  "/api/patient/dismiss-onboarding",
  "/api/patient/previous-therapists",
  "/api/patient/condition-profile/export",
];
console.table(await Promise.all(routes.map(async (route) => {
  const r = await fetch(route, {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: "{}",
  });
  return { route, status: r.status, body: (await r.text()).slice(0, 80) };
})));
```

Repeat signed in as the **hospital**.

Expect. Refused on every one, in both runs. The sharpest of the three is the export: a 200 there hands a non-patient a typeset PDF of an empty health record **named after them**.

And check the suspended case too. Suspend that therapist, then re-run. Still refused - a suspended account must not keep acting as itself.

Step 12.13 - Checkpoint

	Should be
Four dashboards	Each leading with its own figure, naming itself twice, every action reachable
Route sweeps	403 across all three limited desks; 401/403 anonymously
Malformed bodies	4xx, never 500
Impersonation	One session, recorded with a reason, exited
Logs	Read, exported, clear refused inside 30 days
Rate limit	A 429 seen, worded without numbers or blame

14. Part 13 - The site itself, and the small things

What this part does. The cross-cutting checks that do not belong to one journey: the public pages, what happens on a phone, what happens when something goes wrong, and whether the app ever leaves somebody staring at a screen that is not responding.

Time. About 45 minutes.

Step 13.1 - Walk the eight public pages

Do this. Signed out, open each of `/`, `/conditions`, `/how-it-works`, `/home-visit`, `/team`, `/mission`, `/faq`, `/hospitals`.

Expect, on every one

- The same shape: a hero with a photograph, some bands, a "where to go next" index, and the **same closing band** - so wherever a visitor stops reading, the next step is in the same place.
- **The closing band carries a photograph**, not a wall of text on a dark panel, and it is a picture of **somebody booking**, not of treatment. The little confirmation chip over it labels itself an example.
- Every photograph has a **face**, and a **screen in frame** - a laptop, a tablet or a phone. The two exceptions are the home-visit images, which show hands-on treatment, and the clinician reading a scan, who is concentrating rather than smiling.
- The text is **short**. A hero sentence is about a dozen words; a band's lede is under ten and is often absent because the heading already said it. A band with two paragraphs of prose is a **P3** against the word budget.
- A **section rail** down the side lists what is on the page, and the scroll arrow walks it **top to bottom**. An entry that sends the arrow backwards means the rail is out of order with the page.

Then check the index is consistent. Every page's "where to go next" band ends on **booking**, and lists the other pages minus itself.

Step 13.2 - Check the catalogue reads the same everywhere

Do this. Compare one programme card in three places: `/conditions`, its **View full details** dialog, and the patient dashboard's **Book a Session** screen.

Expect

- The **same card**, with cover, chips, ticks, price and a Book button. The dashboard's booking screen must **not** be a plainer text-only list - a patient who has already signed up meeting a plainer catalogue than a stranger does is a **P2**.
- In the dialog, the photograph gets the full width with **nothing written over it** and the heading on its own band **below**. Text over a photograph needs a scrim dark enough for any image, which is why it moved.
- A card with **no** photograph shows the shared tinted placeholder at the **same height** - it must look like a photo that has not been chosen yet, never like one that failed to load.

And check the tap targets. The card body opens the dialog; the **Book** link sits below it and again at the foot of the dialog. A booking link nested *inside* the card's own button is invalid markup and behaves differently per browser - **P2**.

Step 13.3 - Check an admin edit reaches the public site immediately

Do this. With `/conditions` open in one tab, as the admin: rename a condition, change its price, and reorder two of them with **Save order**. Reload the public tab **at once**.

Expect

- Every change is **already there**. These pages are cached for five minutes, and each of those saves is supposed to clear that cache. A change that only appears after a wait reads as a save that silently failed, and is how the same edit gets made twice - **P2**.

- The reorder **survives a reload** of the admin screen too. A reorder that reverts is the pairwise-swap defect, and it used to happen whenever two conditions shared an order number.
- While there are unsaved moves, the screen says `Not saved yet - the public pages still show the old order.` with an **Undo changes** link, and **Save order** is visible but disabled until something has actually moved.

Then the conflict. With two admin tabs open, add a condition in tab B and press **Save order** in tab A.

Expect. Refused with `The condition list changed while you were reordering it. Refresh and try again.` Renumbering a subset collides with the rows it never saw.

And suspend a therapist, then reload `/team`.

Expect. They are gone **immediately**. Their "hide from /team" control is also **disabled** while they are suspended, naming that as the reason rather than offering an action that would change nothing.

Step 13.4 - Delete a condition, three ways

Do this

1. Create `QA Spare Condition` with nothing booked under it. Delete it.
2. Try to delete `QA Back & Spine Care`, which has sessions and a purchase.
3. Delete every session under some condition but leave one **home-visit package** filed against it. Delete it again.
4. With two tabs open, delete a condition in one and press Delete on the same row in the other.
5. Press Delete and drop the network mid-request.

Expect

Step	Expect
1	Deletes. The row leaves the list, and <code>/</code> and <code>/conditions</code> update at once.
2	A dialog - not an eleven-pixel line clipped beside the button - naming the counts and offering turn it off instead , with what that preserves.
3	Names the home-visit package specifically. "It has bookings" would send you to delete sessions and be refused a second time by something you were never told about.
4	<code>That condition has already been deleted. Refresh to see the current list.</code>
5	<code>Could not reach the server. Nothing has changed.</code>

Never a success on a delete that removed nothing. If the row is still listed after a success and a refresh, that is a **P0** - it is the exact defect this check was written for.

And check the log names the condition's title, not "Treatment category". The row is gone by then, so the log is the only thing left that can say which one it was.

Step 13.5 - Check the app always says it is working

Do this. Throttle the network to **Slow 3G** and, on each dashboard, tap something that changes data - approve an account, save a setting, assign a therapist.

Expect

- A **teal bar** above the chrome while the work is in flight. It never shows a percentage - nothing in a browser knows how far along a server render is, and a bar sitting at 90% is a lie people learn to ignore.
- It appears only after a short delay, so a fast action does not flash it.
- The button releases when **its own request** finishes; the bar carries the rest. A button that stays disabled and spinning through a full screen rebuild reads as a hang - **P2**.
- Moving between **sidebar sections** on the patient, therapist and hospital dashboards also draws the bar. Those are plain links, so nothing in React learns the navigation started - if there is no bar there, somebody is sitting on the old screen with no acknowledgement at all.
- With **reduced motion** switched on in the operating system, the bar still appears and simply does not travel. Somebody who asked for less movement still needs to know the app is thinking.

Then check a toggle says what it did. Change any setting.

Expect. A confirmation naming **the thing and its new state** - *"Home visits are on"* - never a bare "Saved". A confirmation that does not name the thing tells somebody a request finished, which they could already see.

Step 13.6 - Check the admin dashboard does not move under you

Do this. Open the admin dashboard and leave it. In a second browser, have a patient book a session. Watch the first screen **without reloading**.

Expect

- The screen **does not move**. Instead the header's **Refresh** button turns teal and carries **the number of changes waiting**.
- Tapping it fetches everything and clears the count.
- **Your own** actions never raise the count - approve something and watch for thirty seconds: no phantom "1 waiting" arrives for the row you just changed or for the log entry it wrote.
- A burst of ten bookings collapses to a small number, not ten.

The other three dashboards still refresh themselves, which is right - a patient watching for a therapist's suggested time **is** waiting for that row.

Step 13.7 - Break things on purpose

Do this

Do	Expect
Open a dashboard URL with a nonsense id in it	A friendly error screen, not a stack trace. Only an opaque reference code is shown - an error message can carry a column name or a row id, and patients see these screens.
Drop the network mid-save anywhere	A readable message saying nothing changed, not a silent failure and not a blank screen
Open <code>/admin/dashboard?section=money&tab=payouts</code> as the Master Admin	It server-renders Payouts directly - it must not paint Today first and jump

Do	Expect
Open the same link with a nonsense tab key	It lands on that section's first screen, and an amber line says so. A silent redirect looks like it worked.

Step 13.8 - Use the whole thing on a phone

Do this. On a real phone, or at a 390px-wide window, walk: the home page, `/book` end to end, the patient dashboard's every screen, and the admin dashboard.

Expect

- Nothing scrolls sideways.
- Tap targets are finger-sized. The **body map** in the Pain Map stacks front and back rather than shrinking both until neither can be tapped.
- The sidebars become a **drawer**, and **Back to Home** is in it.
- Dialogs are usable - they fill the screen rather than sitting half off it.
- The **Razorpay sheet** works.
- A dialog opened **from inside another dialog** covers the whole screen, not just the panel it came from. A confirmation that sits at the top of scrolled content and slides away as you scroll is a real defect and has happened here.

Step 13.9 - Check the keyboard and a screen reader can get through

Do this. Put the mouse away. Tab through `/book`, then through a dashboard.

Expect

- Every control is reachable, in a sensible order, with a **visible focus ring**.
- A dialog **moves focus into itself**, traps Tab while open, closes on **Escape**, and returns focus to whatever opened it.
- Every control has a name. Icon-only buttons - the debug bar's page picker and its simulated-time box included - announce something rather than nothing.
- Small grey text on white is readable. Labels, counts, hints and codes in a pale grey are a real accessibility failure, not a taste question - report them as **P2**.
- The two rotating widgets on the home page have **different** names, and the "areas of practice" picker works with arrow keys.
- Only **one** thing moves on screen at a time. The areas-of-practice carousel must never advance by itself while the walkthrough beside it is rotating.

Step 13.10 - Check the splash one more time

Do this

Do	Expect
Open the site in a brand-new tab	The splash shows
Reload	It does not

Do	Expect
Navigate between pages	It does not
Leave the tab in the background past the configured minutes and come back	It shows
Start a checkout, leave the tab for your bank's app, come back	It does not . Splashing over a payment in progress is the one thing this must never do - P1 .
Switch reduced motion on	It is skipped entirely

Then change its settings (Settings → Public Site → Opening Splash): the name line, the wording, the hold, and the away threshold. Set the away threshold to `0`.

Expect. `0` means **first load only**. There is deliberately no value meaning "greet on every tab focus" - that is the setting that would splash over a checkout.

Step 13.11 - The mission, the vision, the promises and the limits

These four blocks are what `/mission` **is**, and all of them are an admin's to write. They used to be constants only a developer could change, which made the copy most likely to be argued over the copy nobody in the clinic could touch.

Do this

1. Open **Settings → Public Site → Mission & Vision**.
2. Replace the mission with `QA mission line for testing, replaced by the admin.` and save.
3. Open `/` and `/mission` **at once**, in another tab.
4. Now **clear the box** entirely and save again.
5. Reload both pages.

Expect

- Step 3: the new sentence is on **both** pages immediately. Both are cached, and saving is supposed to clear that cache - a reworded mission that stays old for five minutes reads as a save that failed.
- The home page quotes the mission and the vision **in full** - they are two sentences, and a paraphrase there would be the home page making a weaker version of the same claim.
- Step 5: **the original wording is back**. Blank is the undo, not an error - it is how an admin restores a sentence they cannot read out of the source.
- The form **warns** past about fifteen words and saves anyway; a genuinely over-long value is refused by a character limit, which is about the card the line renders in rather than about the writing.

Now the promises and the limits. Both are lists of rows on the same screen.

Do this	Expect
Add a promise: title <code>QA promise</code> , a short body, pick an icon	It appears on <code>/mission</code> , and on the home page as a title only , linking through
Reorder them with the arrows, then Save order	Nothing changes until you save; then the new order is live on both pages and survives a reload

Do this	Expect
Switch every promise off	The whole band disappears from <code>/mission</code> - and so does its entry in the section rail . A rail entry pointing at a band that does not render sends the scroll arrow nowhere.
Delete every promise	The shipped wording comes back . That is correct, and the screen says so - otherwise a delete whose visible effect is the original text reappearing reads as a failed delete.
Switch the limits off while the promises stay on	Only the limits band goes. Writing one must never empty the other.
Pick an icon	It is a picker , not a text box. Free text there is a way to put an empty square on the mission page, and a blank box does not say whether the icon or the row failed.

And one wording check. No heading on that page counts the cards - nothing reads "Four things, every patient" over three of them. That is a number somebody forgets to change, and it tells the reader something untrue.

Step 13.12 - Checkpoint

	Should be
Eight public pages	One template, photographs with faces and screens, rail in order
Catalogue	The same card on the public page and the patient's booking screen
Admin edits	Live on the public site immediately
Progress	A bar on all four dashboards, no percentage
Phone	No sideways scroll, drawer nav, dialogs usable
Keyboard	Everything reachable, dialogs trap and restore focus

15. Sign-off

15.1 What the run should have left behind

Check the database matches this before signing anything. A mismatch is either a defect you have not written up or a step you skipped.

	Should be
Conditions	Three, one carrying an uploaded and positioned cover
Programmes	P1 (6 sessions), P2 (8), P3 (1)
Home-visit packages	HV1 (1 visit), HV2 (4)
Service areas	<code>560038</code> at ₹150, <code>560095</code> at ₹200, one waitlist entry for <code>560025</code>
Admins	Master Admin plus Operations, Finance, Clinical

	Should be
Therapists	A (60% / 65%, rostered), B (55%, rostered), C (50%, on leave)
Hospital	QA Sunrise Hospital at 10%, one commission earned
Patients	A, B, C, D, E, F, plus the declined Z
Sessions	At least one delivered video session, one delivered home visit, one cash visit, one refunded, one forfeited
Programme	One purchased, six sessions, at least one still unbooked
Recommendations	One rejected, one approved with changes, one withdrawn, one purchased
Money	A settled payout with cash netted off, three costs, discounts given split by rule
Logs	A row for every admin action above, including the reset and the impersonation

15.2 The sign-off sheet

Build / commit tested	
Environment (URL and Supabase project)	
Tester	
Started	
Finished	
Steps passed	
Steps failed	
Steps blocked	
Steps N/A (and why)	
P0 defects raised	
P1 defects raised	
P2 / P3 defects raised	
Safe to release?	Yes / No / Yes with the listed exceptions
Signature	

The rule for that last row. Any open **P0** is a No. A P0 is money, clinical data or access control being wrong: somebody charged a figure they were not quoted, one patient's record reachable by another, a route answering 200 to a caller it should refuse.

15.3 The ten checks that matter most

If you have time for nothing else, these are the ones whose failure is worst. Each names the step that covers it.

#	Check	Step
1	The payment screen charges exactly the figure it quoted	4.2, 4.4, 8.1, 11.1
2	A free total takes no money at all, and never ₹1	11.6
3	A programme cannot be bought without a recommendation	7.2, 8.3
4	A recommendation the clinic has not approved reaches the patient in no form	7.1, 7.2
5	One patient's clinical record is unreachable by another patient, and by an unassigned clinician	6.6
6	Every admin route refuses the three limited desks	12.2
7	Every mutating route refuses an anonymous caller	12.8
8	A purchase is never rewritten by a later catalogue edit	7.10
9	Travel is paid to the therapist in full and never counted as revenue	8.6, 10.2
10	Suspending an account locks it out at the route, not only on the screen	9.9, 12.7

15.4 What this run deliberately does not cover

Say so explicitly rather than letting silence imply coverage.

- **Real money.** Everything here is Razorpay test mode. Nothing proves a live key works.
- **Email and SMS delivery.** Addresses end `.test` and cannot receive anything. Password-reset and invite emails are checked as far as "the request succeeded", never as "it arrived".
- **Google Calendar and Meet, where the account is not connected.** If System Health says *Not set up*, every Meet check is N/A rather than failed.
- **Load and performance.** One tester on one browser says nothing about fifty patients booking at once.
- **Browsers other than the one you used.** Note which you used on the sheet.
- **The database's own guards.** Several rules are enforced by the database as well as by a route - the credit ledger's append-only trigger, the retention floor, the reorder function's completeness check. Proving those needs SQL against a scratch database, which is a developer's job and lives in the repository's own checks.
- **Anything a second run would need reset.** See below.

15.5 Running it a second time

This document builds its own data, which makes it repeatable - but only from the same starting point.

The clean way. Go back to **Step 1.2** and reset. Everything from Part 2 onward rebuilds itself, and that is the point of the ordering.

What you lose by not resetting. Running Part 4 again against a database that already has Patient A gives you *"user already registered"* at Step 4.2, which is correct behaviour and reads as a broken signup. The same is true of every unique fixture: the hospital's email, the promo code, the conditions' names.

What to write down before you reset, because the wipe takes it:

- The three scoped admins' generated passwords - they are shown once and stored nowhere you can reach.
- The hospital's generated password and referral code.

- Any defect you have raised but not yet written up, with its screenshots.

What the reset keeps: admin logins, and your conditions with their programmes. Everything else in Part 2 - home-visit packages, service areas, FAQs, testimonials - you create again.

15.6 If something in this document is wrong

This run quotes real screen names, real settings, real error strings and real prices. When the application changes, some of them go stale, and a stale expectation reads exactly like a defect.

Two things separate one from the other:

1. **Check the step's own reasoning.** Nearly every expectation here says *why*. If the reason still holds and only the wording has moved, it is the document that is stale. If the reason no longer holds, it is a product change somebody should have documented - raise it either way.
2. **Check which server you are pointed at.** The public pages are cached, so a production build serves markup generated before your fixtures existed. A condition you just created being absent from `/conditions` on a production build is that cache, not a broken catalogue.

Raise a documentation fix the same way you raise a defect, marked **Doc**. It costs a line and saves the next tester an hour.
